

搜索求解2

主讲：刘夏雷、郭春乐、王亚星

南开大学计算机学院

<https://mmcheng.net/xliu/>

致谢：本课件主要内容来自浙江大学吴飞教授、
南开大学程明明教授

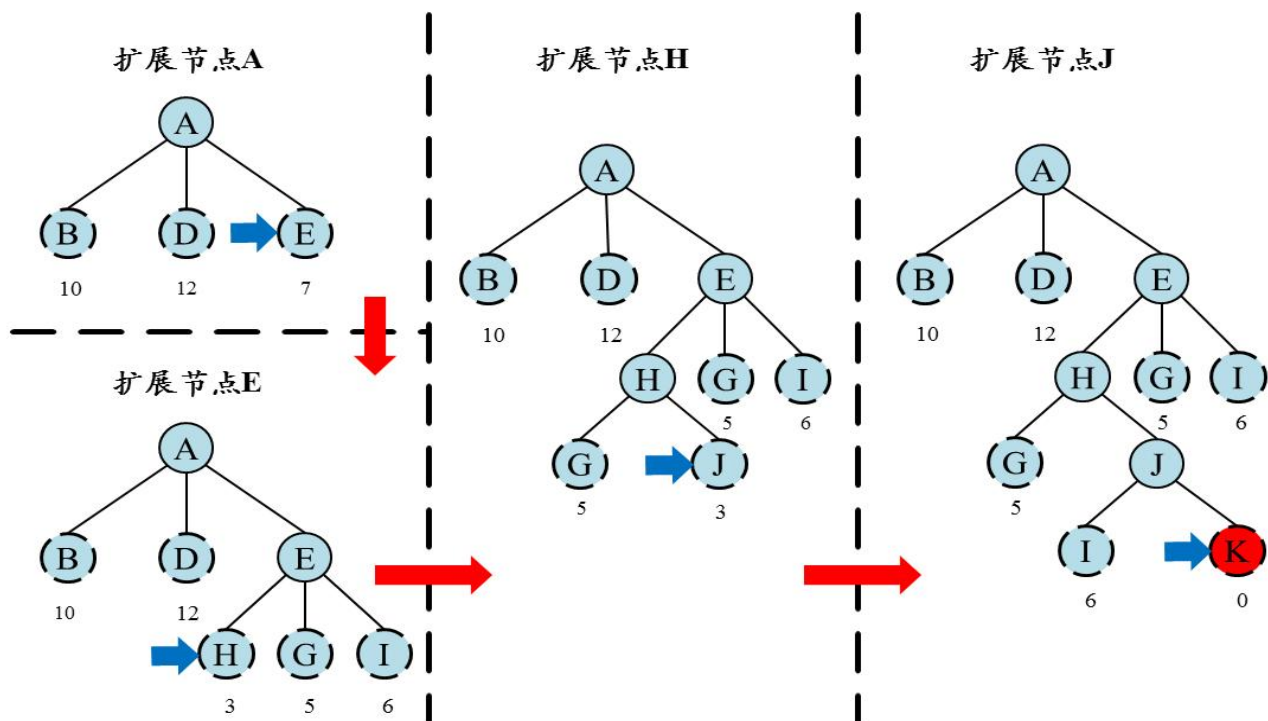
提纲

- 搜索算法基础
- 启发式搜索
- 对抗搜索
- 蒙特卡洛树搜索

内容回顾：贪婪最佳优先搜索

- 贪婪最佳优先搜索：评价函数 $f(n)$ =启发函数 $h(n)$

- 例：启发函数为任意一个城市与终点城市K之间的直线距离



算法找到了一条从起始结点到终点结点的路径 $A \rightarrow E \rightarrow H \rightarrow J \rightarrow K$ ，但这条路径并不是最短路径，实际上最短路径为 $A \rightarrow E \rightarrow G \rightarrow K$ 。

贪婪最佳优先搜索的过程

内容回顾：A*算法

- 评价函数： $f(n) = g(n) + h(n)$
 - $g(n)$ 表示从起始结点到结点 n 的开销代价值
 - $h(n)$ 表示从结点 n 到目标结点路径中所估算的最小开销代价值
 - $f(n)$ 可视为经过结点 n 、具有最小开销代价值的路径。

$$\underbrace{f(n)}_{\text{评价函数}} = \underbrace{g(n)}_{\substack{\text{起始结点到结点}n\text{代价} \\ \text{(当前最小代价)}}} + \underbrace{h(n)}_{\substack{\text{结点}n\text{到目标结点代价} \\ \text{(后续估计最小代价)}}}$$

作业解答

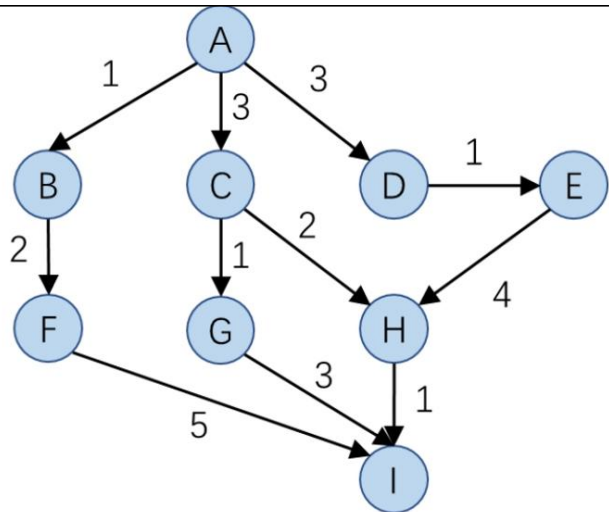
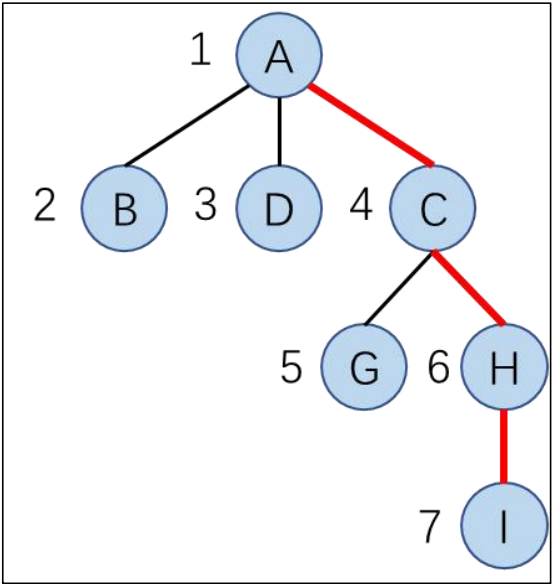
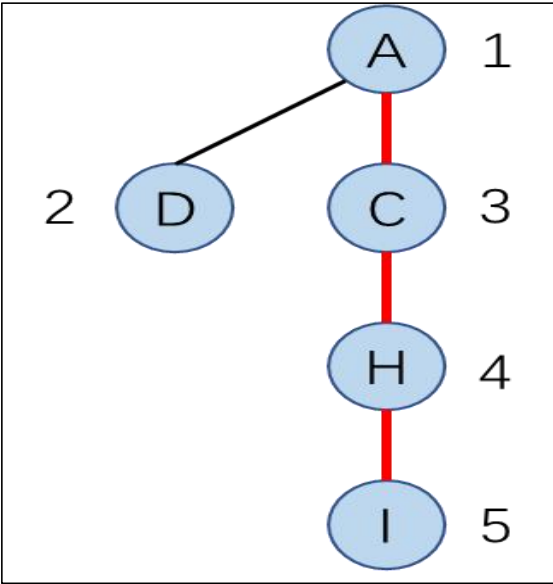


图1 状态转移图

1. 考虑图1中的问题，给定每个状态的启发函数如下表所示。若仍以状态A为初始状态、状态I为终止状态，请分别使用以下算法求解从A到I的路径，请画出算法找到第一条路径时的搜索树，并在搜索书中标出结点的扩展顺序，以及找到的路径。（若有多个节点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的节点）。

- (1) 基于树搜索的贪婪最佳优先搜索。
- (2) 基于图搜索的A*算法。

状态	A	B	C	D	E	F	G	H	I
启发函数	5	4	3	2	5	5	2	1	0



作业解答


南开大学
 NANKAI UNIVERSITY
 94 Weijin Road, Tianjin
 People's Republic of China

解: (1) 初始结点: A 扩展结点: B, C, D 选择: D
 (2) 初始结点: A 扩展结点: B, C, D 选择: C
 (3) 初始结点: A 扩展结点: B, C, D 选择: H
 (4) 初始结点: A 扩展结点: B, C, D 选择: I

路径: A-C-H-I
 权: 1 (A) 2 (D) 3 (C) 4 (H) 5 (I)

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

(1) A 选D A 选C A 选H A
 B C D B C D B C D B C D
 4 3 2 4 3 4 1 1 4 1 1
 E G H E G H E
 2 1 5 2 1 5

最终结果: A ①
 D ②
 C ③
 H ④
 I ⑤

(2) A 选B A 选D A
 B C D B C D B C D
 4 4 5 4 3 6 4 3 6
 F F F
 2 4 5 2 4 5 2 4 5

最终结果: A ①
 B ②
 D ③
 C ④
 H ⑤
 I ⑥

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

或者: 1 (A) 2 (B) 3 (C) 4 (D) 5 (E)

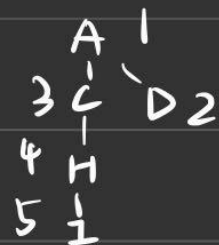
作业解答

(1) 与 A 连接的 B、C、D 中, D 的启发函数最小

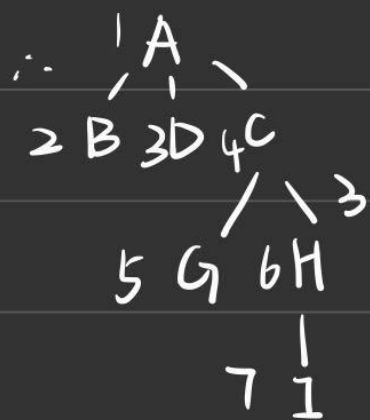
B、C、E 中, C 的启发函数最小

G、H、E 中, H 的启发函数最小

I 的启发函数为 0



(2) $f(B) = 1 + 4 = 5$ $f(C) = 3 + 3 = 6$ $f(D) = 3 + 2 = 5$ $B < D \therefore B$ 是 2 $f(F) = 2 + 5 = 7$



D 是 3 $f(E) = 1 + 5 = 6$ $C < E \therefore C$ 是 4

$f(G) = 1 + 2 = 3$ $f(H) = 2 + 1 = 3 \therefore G < H \therefore G$ 是 5 $f(I) = 3 + 0 = 3$

$H < I \therefore H$ 是 6 $H(I) = 1 + 0 = 1 \therefore I$ 是 7

对抗搜索

- 对抗搜索(Adversarial Search)也称为博弈搜索(Game Search)
- 在一个竞争的环境中，智能体(agents)之间通过竞争实现相反的利益，一方**最大化**这个利益，另外一方**最小化**这个利益。



对抗搜索：主要内容

- **最小最大搜索(Minimax Search)**

- 最小最大搜索是在对抗搜索中最为基本的一种让玩家来计算最优策略的方法

- **Alpha-Beta剪枝搜索(Pruning Search)**

- 一种对最小最大搜索进行改进的算法，即在搜索过程中可剪除无需搜索的分支节点，且不影响搜索结果。

- **蒙特卡洛树搜索(Monte-Carlo Tree Search)**

- 通过采样而非穷举方法来实现搜索。

对抗搜索

- 本课程目前主要讨论在确定的、全局可观察的、竞争对手轮流行动、零和游戏(zero-sum)下的对抗搜索
 - 所谓零和博弈是博弈论的一个概念，属非合作博弈。指参与博弈的各方，在严格竞争下，一方的收益必然意味着另一方的损失，博弈各方的收益和损失相加总和永远为“零”，双方不存在合作的可能。与“零和”对应，“双赢博弈”的基本理论就是“利己”不“损人”，通过谈判、合作达到皆大欢喜的结果。

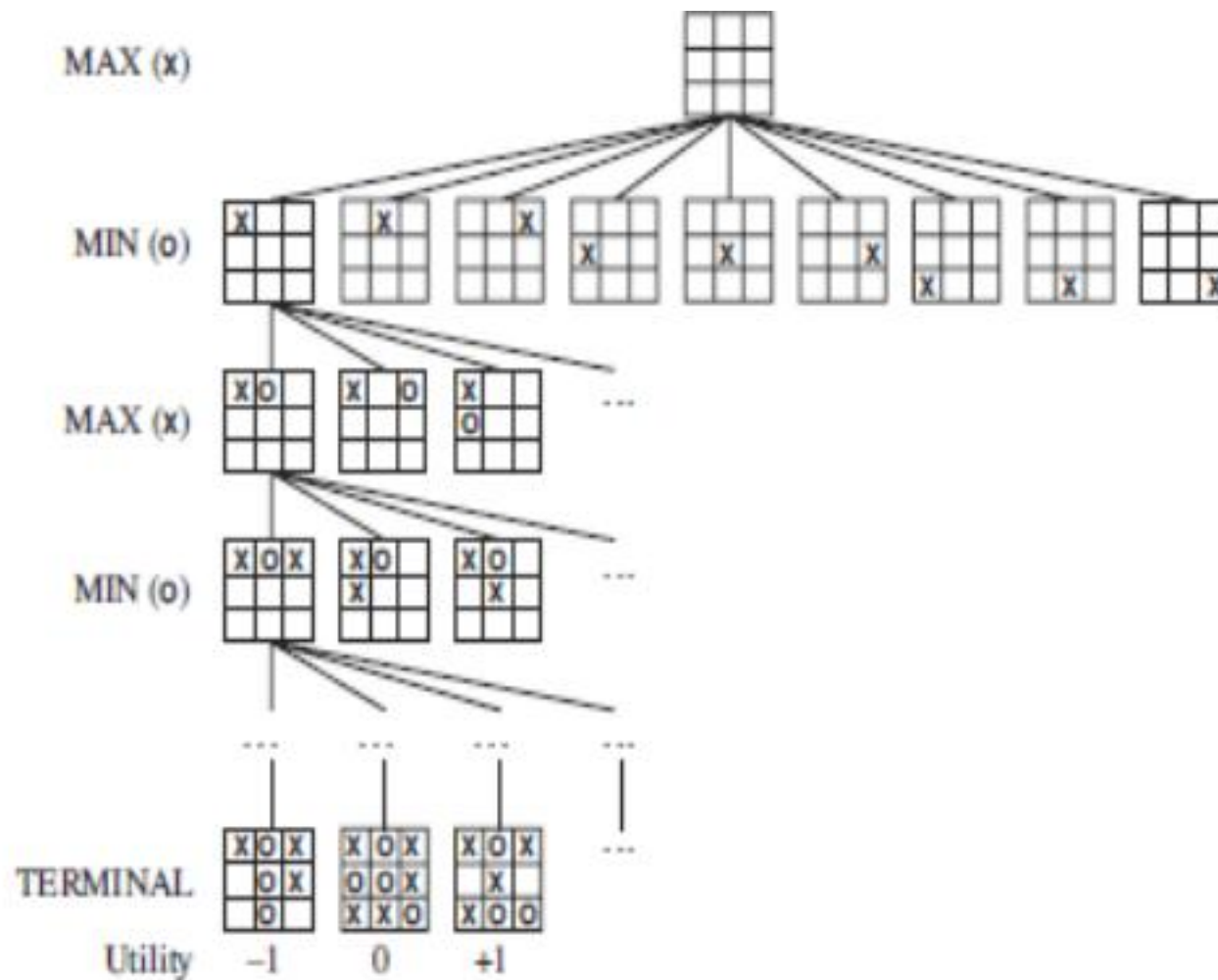
对抗搜索

- 两人对决游戏 (MAX and MIN, MAX先走) 可如下形式化描述, 从而将其转换为对抗搜索问题

初始状态 S_0	游戏所处的初始状态
玩家 $PLAYER(s)$	在当前状态 s 下, 该由哪个玩家采取行动
行动 $ACTIONS(s)$	在当前状态 s 下所采取的可能移动
状态转移模型 $RESULT(s, a)$	在当前状态 s 下采取行动 a 后得到的结果
终局状态检测 $TERMINAL - TEST(s)$	检测游戏在状态 s 是否结束
终局得分 $UTILITY(s, p)$	在终局状态 s 时, 玩家 p 的得分。

对抗搜索

- **Tic-Tac-Toe游戏的对抗搜索**
 - MAX先行，可在初始状态的9个空格中任意放一个X
 - MAX希望游戏终局得分高、MIN希望游戏终局得分低
 - 所形成游戏树的叶子结点有 $9! = 362,880$, 国际象棋的叶子节点数为 10^{40}



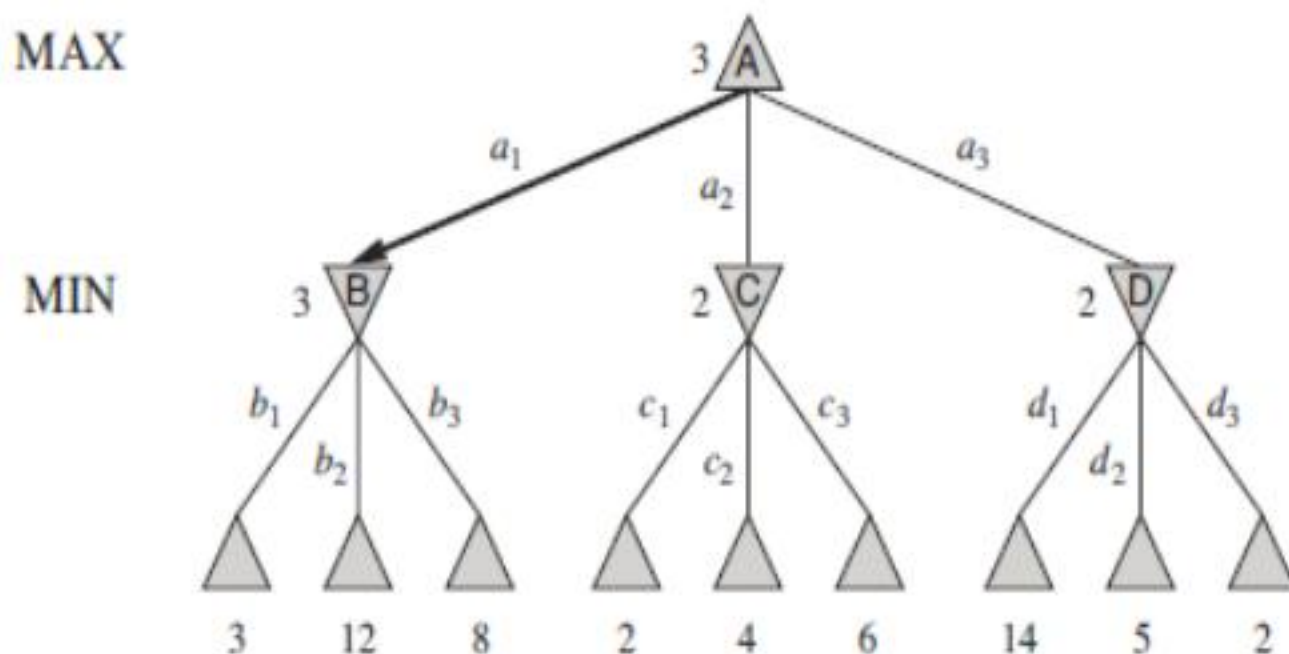
Tic-Tac-Toe中部分搜索树

对抗搜索：minimax算法

- 给定一个游戏搜索树，minimax算法通过每个节点的minimax值来决定最优策略。
 - MAX希望最大化minimax值，而MIN则相反

对抗搜索：minimax算法

- 给定一个游戏搜索树，minimax算法通过每个节点的minimax值来决定最优策略。
- 通过minimax算法，我们知道，对于MAX而言采取 a_1 行动是最佳选择，因为这能够得到最大minimax值(收益最大)。



对抗搜索：minimax算法

- 优点:

- 算法是一种简单有效的对抗搜索手段
- 在对手也“尽力而为”前提下，算法可返回最优结果

- 缺点:

- 如果搜索树极大，则无法在有效时间内返回结果

- 改善:

- 使用alpha-beta pruning算法来减少搜索节点
- 对节点进行采样、而非逐一搜索 (i.e., MCTS)

对抗搜索：Alpha-Beta 剪枝搜索

- 在极小化极大算法(minimax算法)中减少所搜索的搜索树节点数。该算法和极小化极大算法所得结论相同，但剪去了不影响最终结果的搜索分枝。

$MINIMAX(root)$

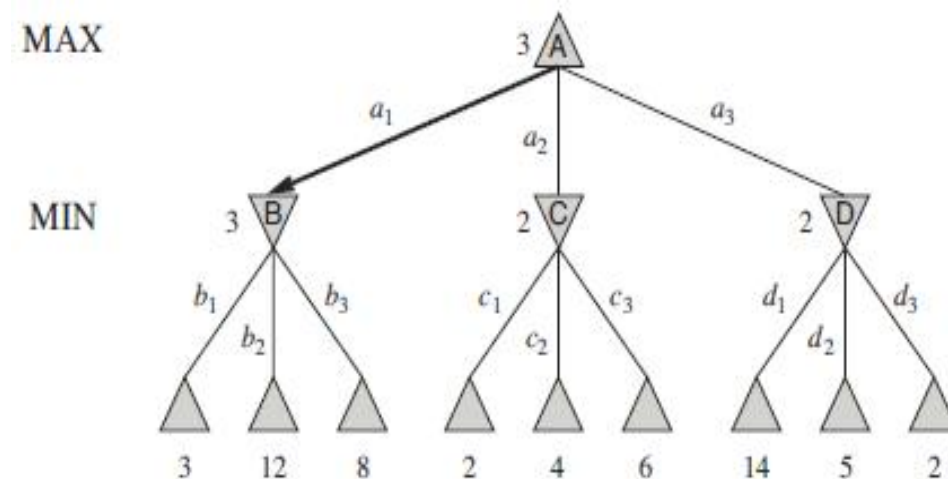
$= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2))$

$= \max(3, \min(2, x, y), 2)$

$= \max(3, z, 2) = 3$

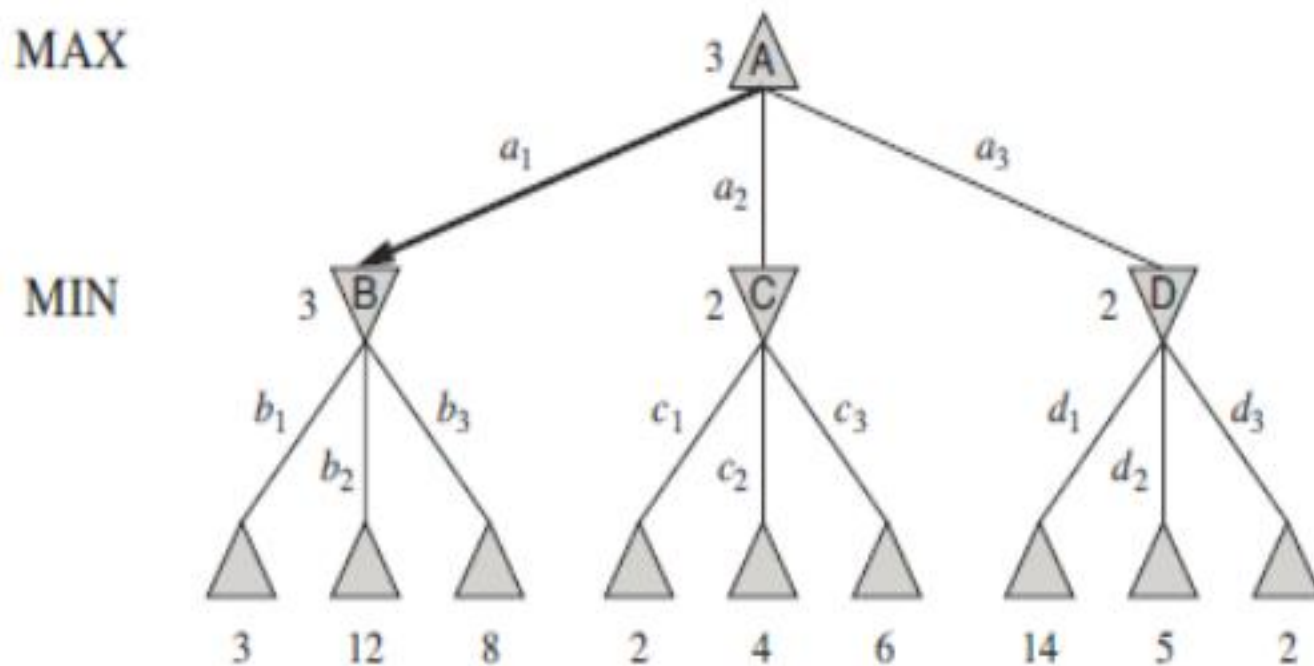
where $z = \min(2, x, y) \leq 2$

可以看出：根节点(即 MAX 选手)的选择与 x 和 y 两个值无关(因此， x 和 y 可以被剪枝去除)

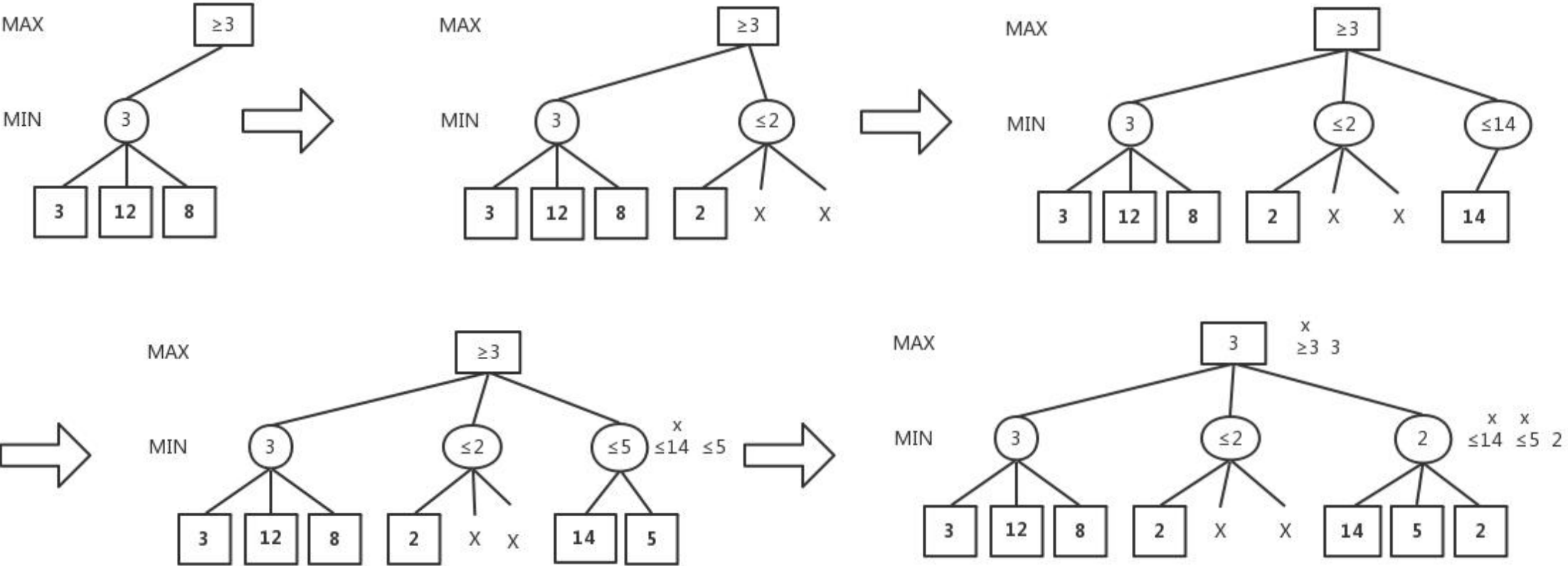


对抗搜索：Alpha-Beta 剪枝搜索

图中MIN选手所在的节点C下属分支4和6与根节点最终优化决策的取值无关，可不被访问。



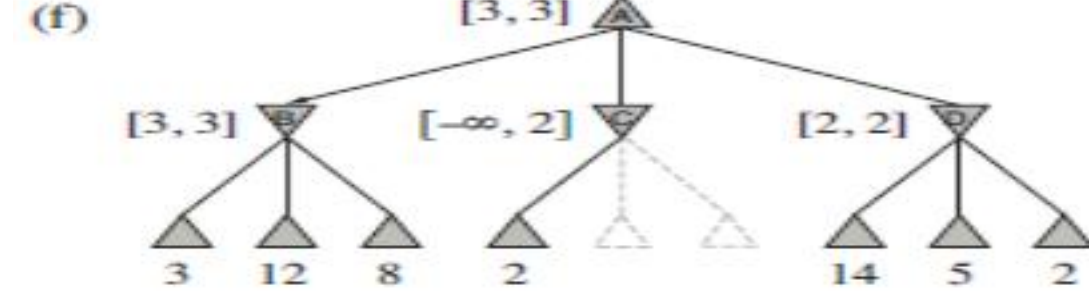
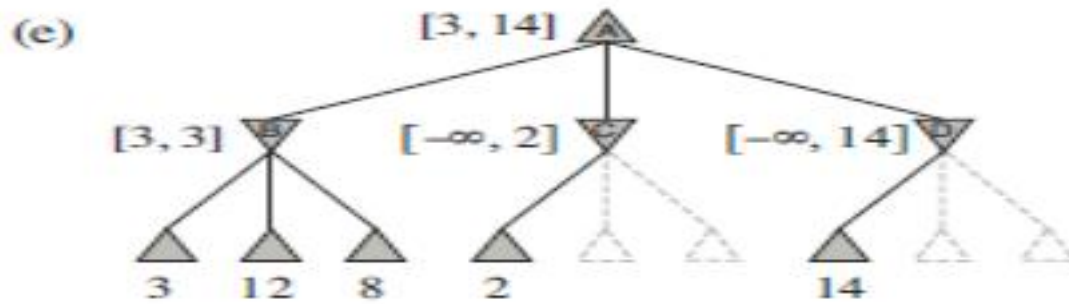
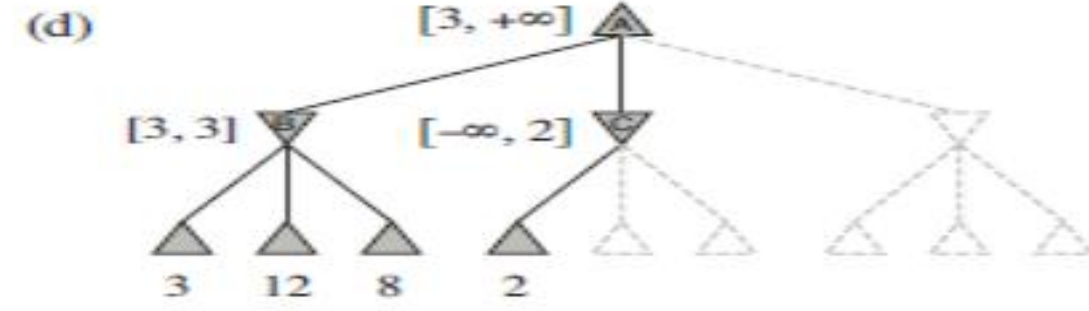
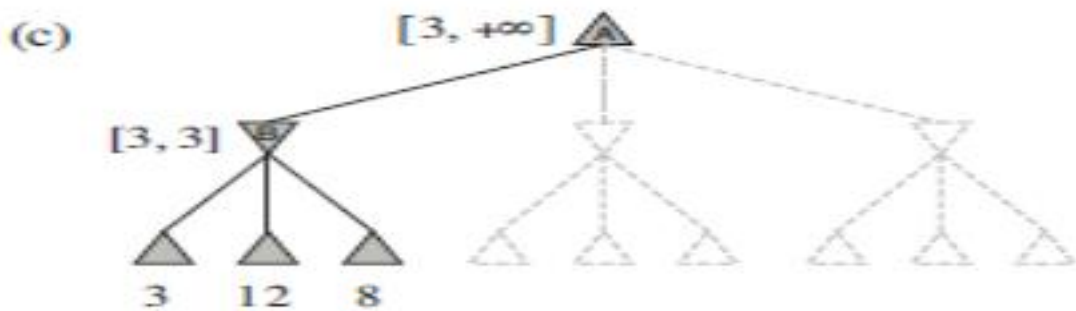
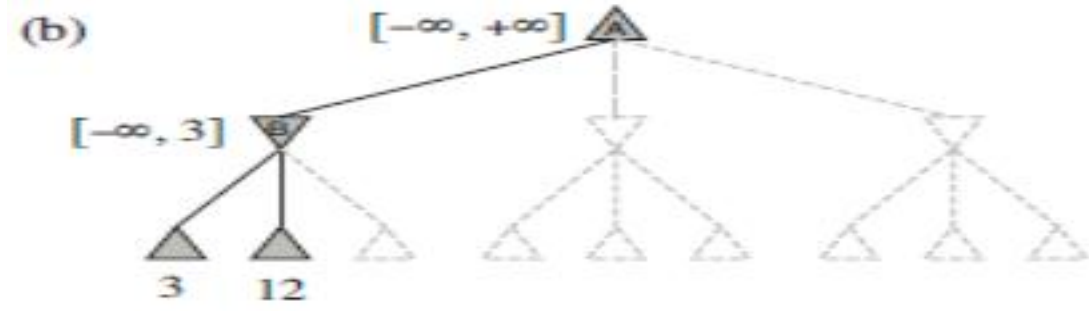
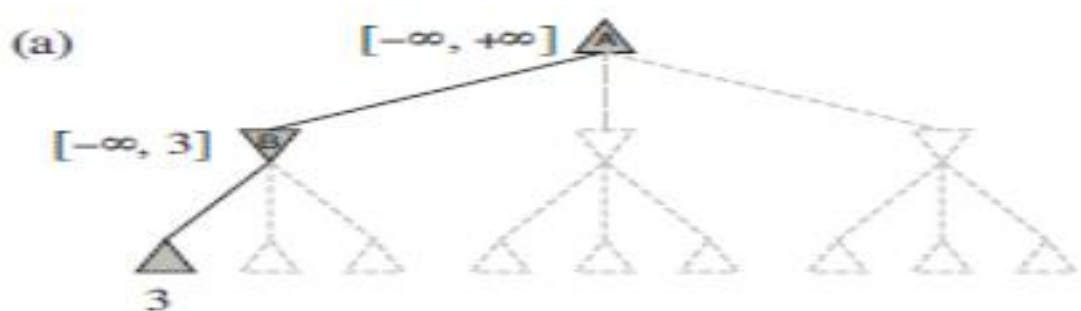
对抗搜索：Alpha-Beta 剪枝搜索



Alpha值(α)	MAX节点目前得到的最高收益
Beta值(β)	MIN节点目前可给对手的最小收益
α 和 β 的值初始化分别设置为 $-\infty$ 和 ∞	

对抗搜索：Alpha-Beta 剪枝搜索

- 从 α 和 β 的变化来理解剪枝过程

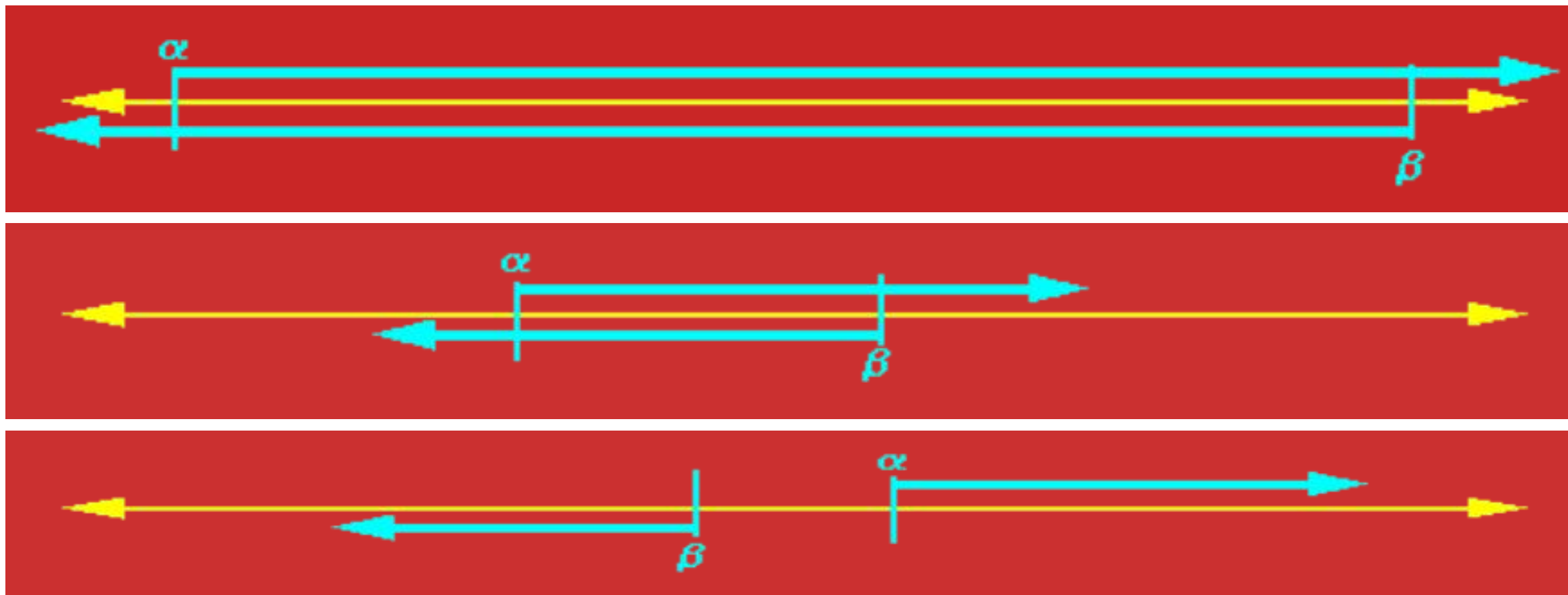


对抗搜索：如何利用Alpha-Beta 剪枝

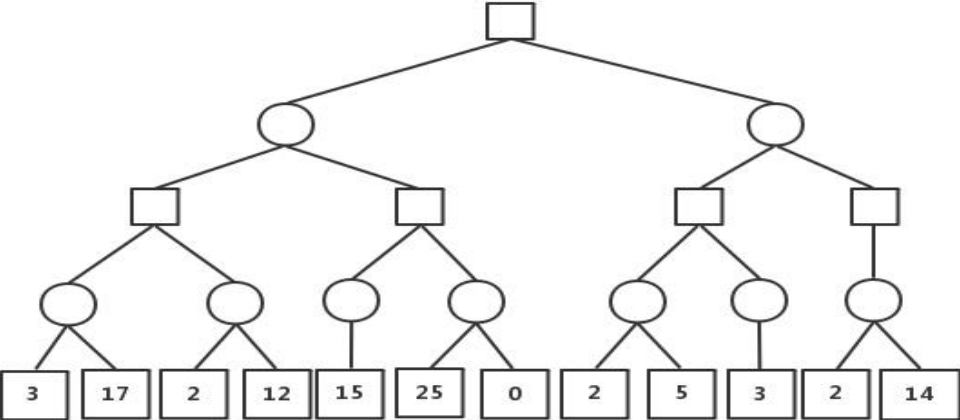
- α 为可能解法的最小下界
- β 为可能解法的最大上界
 - 如果节点 N 是可能解法路径中的一个节点，则其产生的收益一定满足如下条件： $\alpha \leq reward(N) \leq \beta$ (其中 $reward(N)$ 是节点 N 产生的收益)
 - 每个节点有两个值，分别是 α 和 β 。节点的 α 和 β 值在搜索过程中不断变化。其中， α 从负无穷大($-\infty$)逐渐增加、 β 从正无穷大(∞)逐渐减少。如果一个节点中 $\alpha > \beta$ ，则该节点的后续节点可剪枝。

对抗搜索：如何利用Alpha-Beta 剪枝

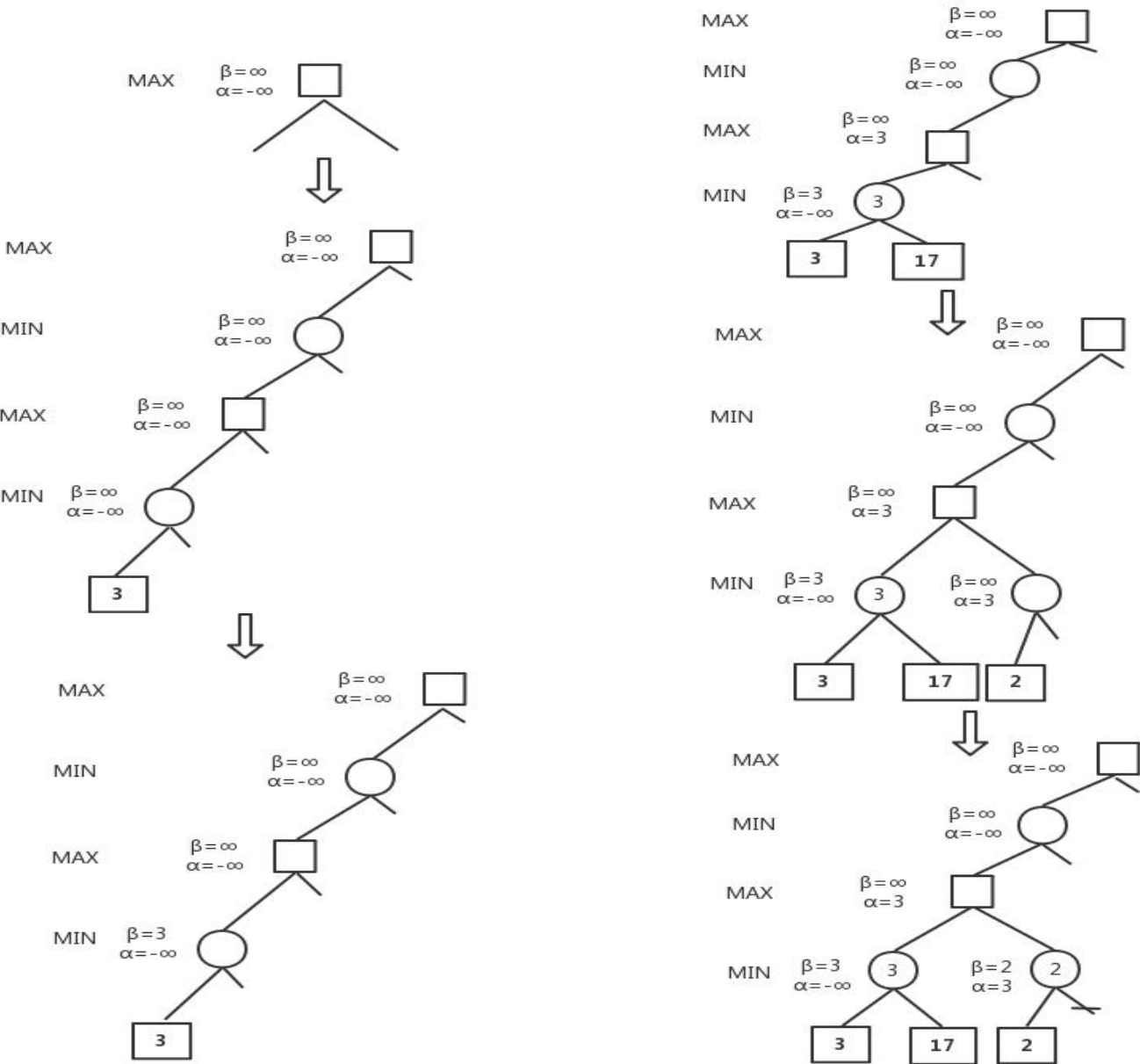
- α 为可能解法的最小下界
- β 为可能解法的最大上界



对抗搜索：Alpha-Beta 剪枝搜索示意

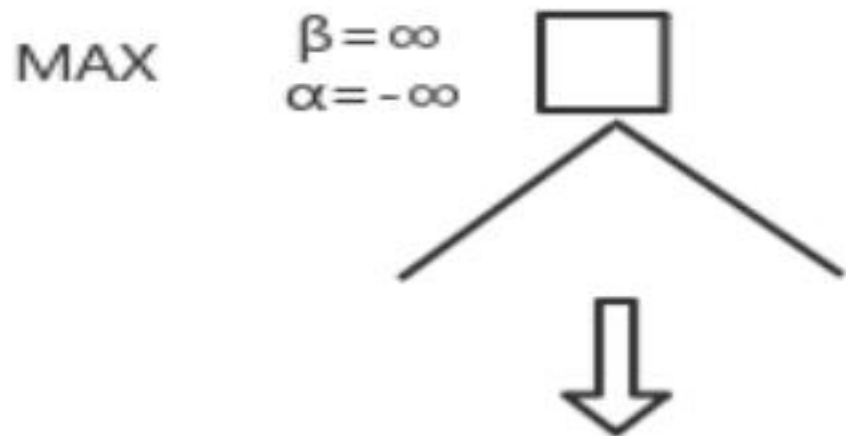


没有必要枚举，可以先继承父节点的 α, β ，再按规则更新。



对抗搜索：Alpha-Beta 剪枝搜索示意

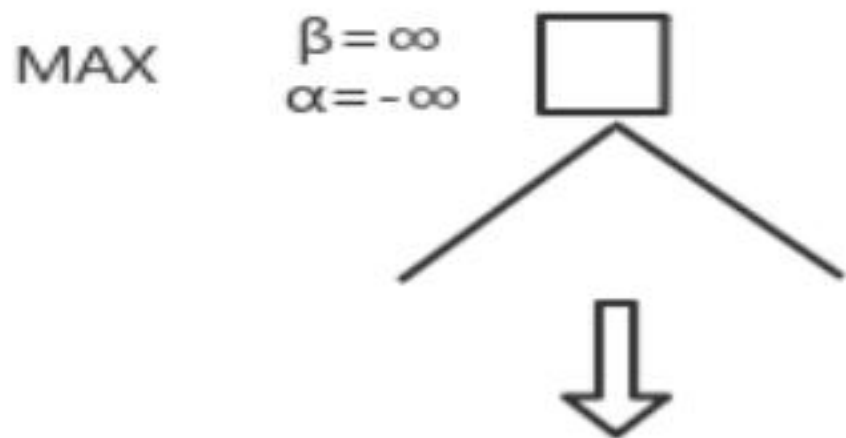
1



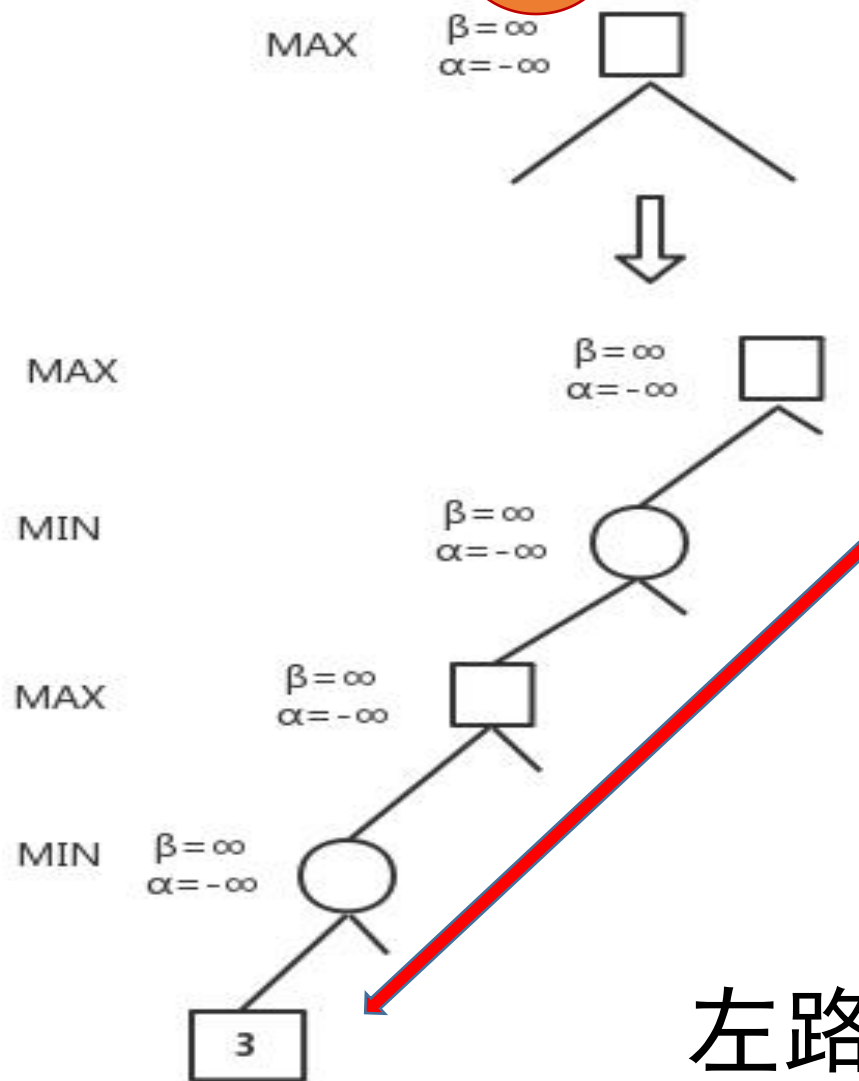
α, β 赋值

对抗搜索：Alpha-Beta 剪枝搜索示意

1



2

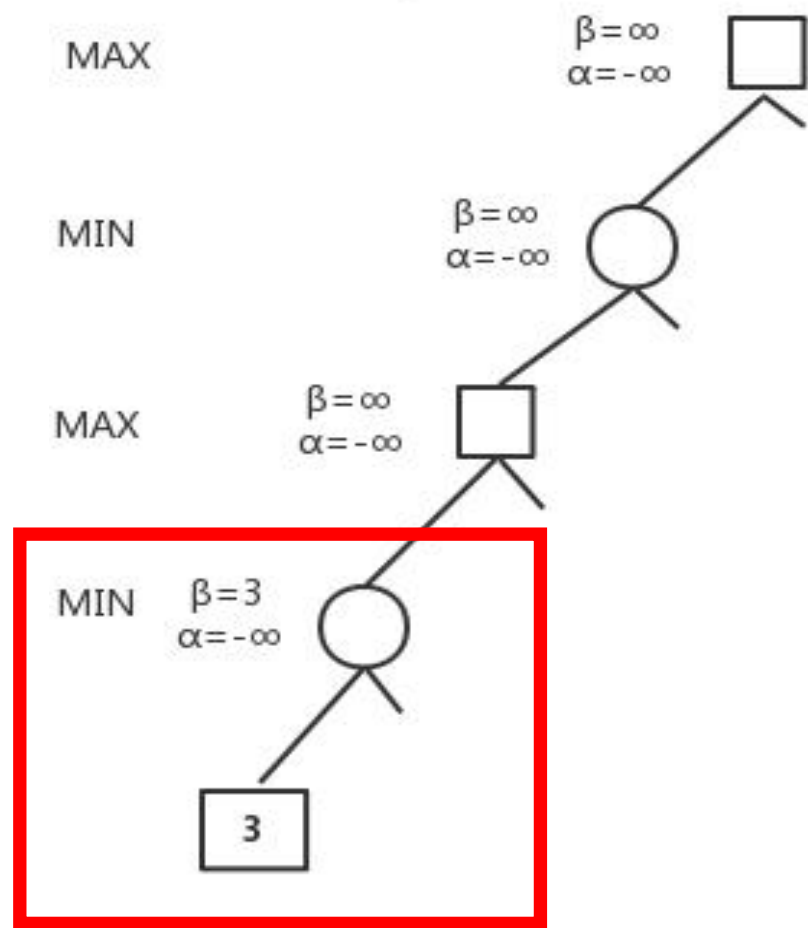


α, β 赋值

左路扩展到底

对抗搜索：Alpha-Beta 剪枝搜索示意

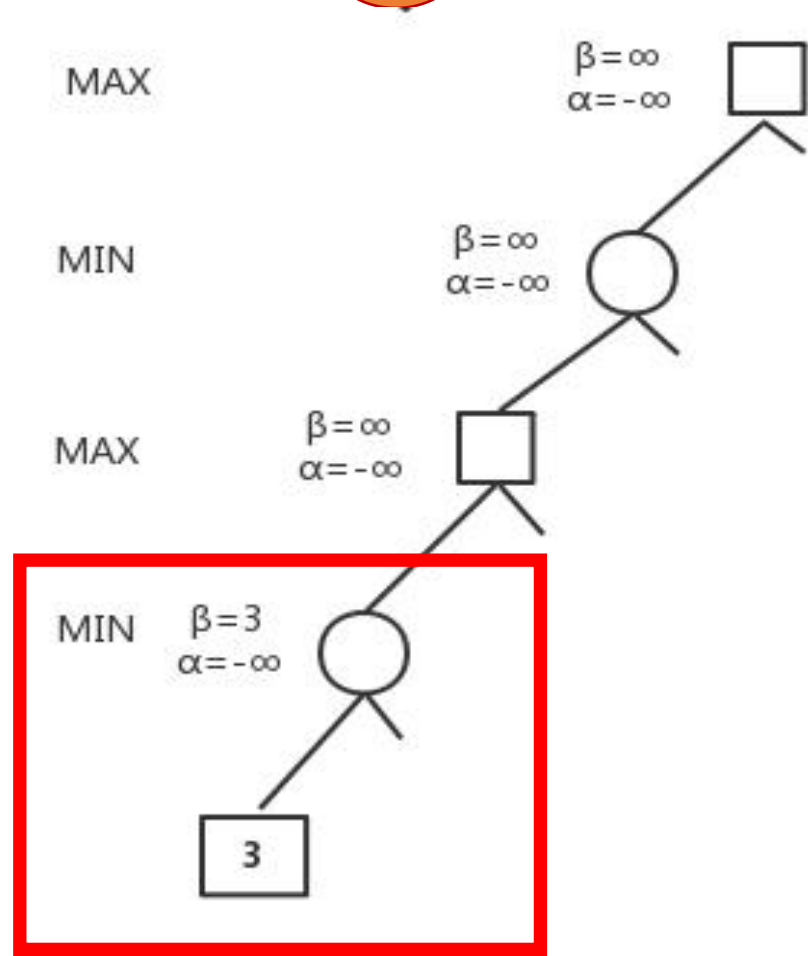
3



*MIN*节点, $\beta = 3$

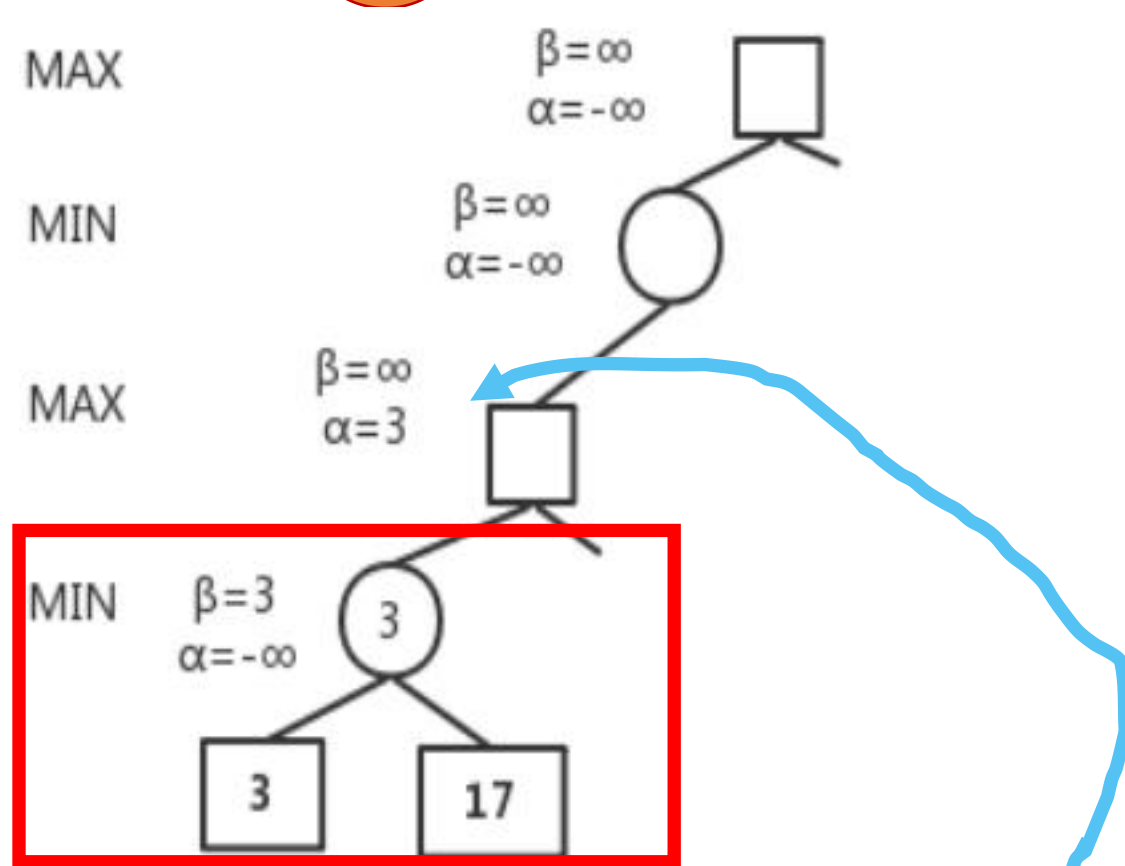
对抗搜索：Alpha-Beta 剪枝搜索示意

3



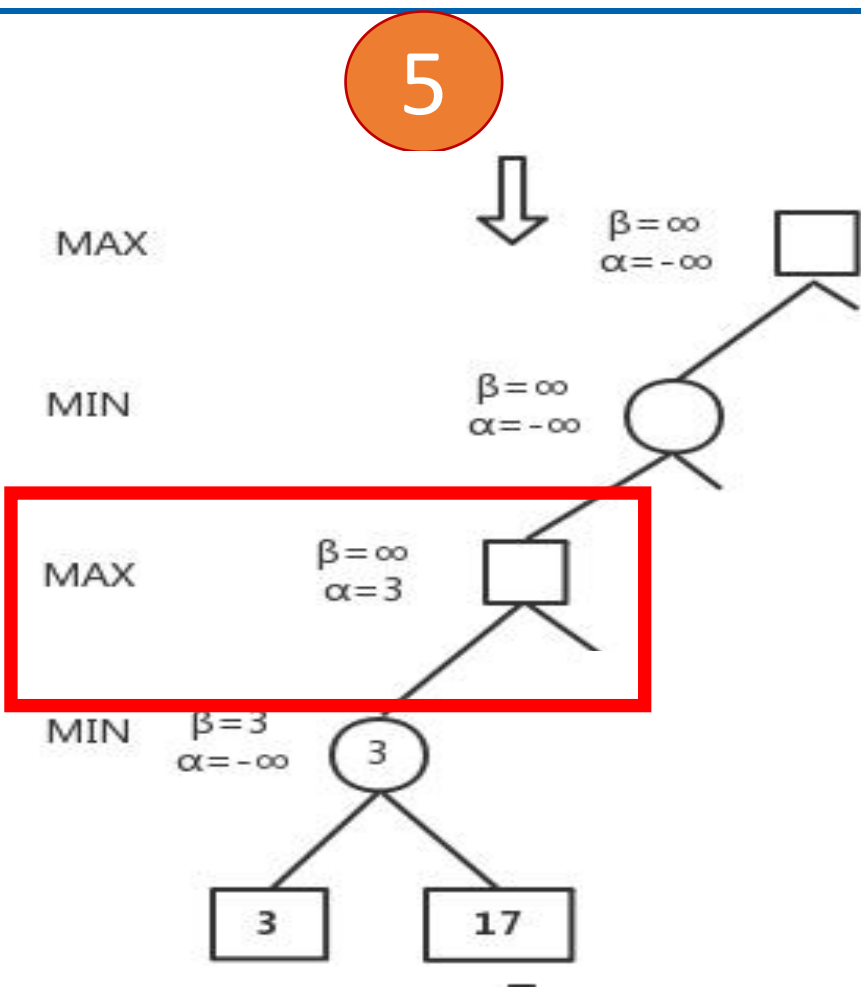
MIN节点, $\beta = 3$

4



MIN节点, β 仍为3。
完毕, MAX节点, $\alpha=3$

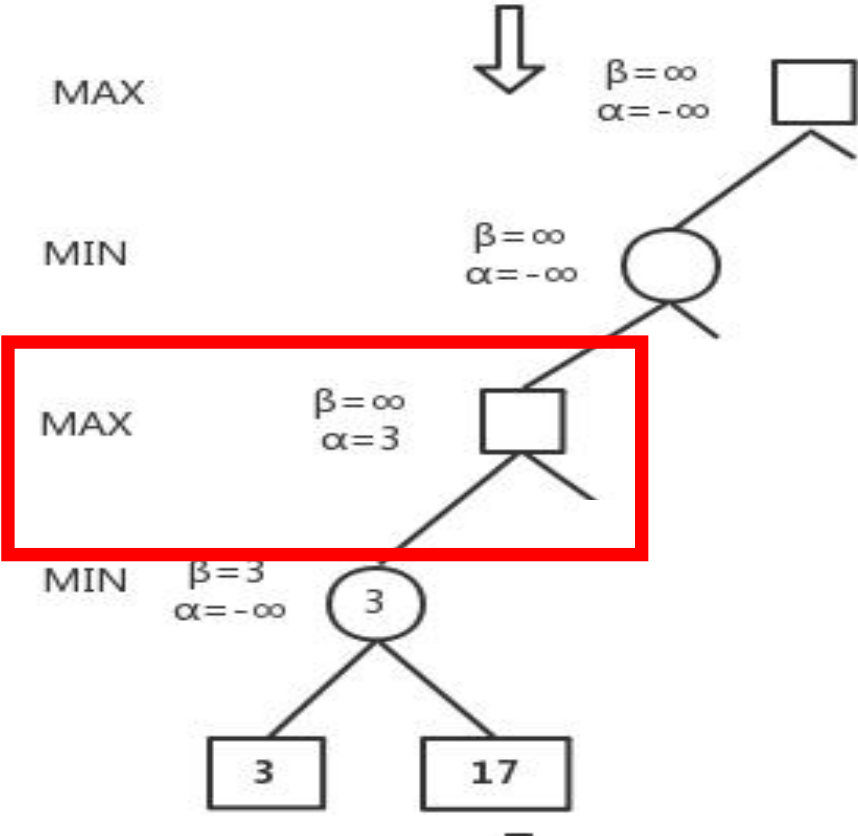
对抗搜索：Alpha-Beta 剪枝搜索示意



MAX 节点, $\alpha = 3$

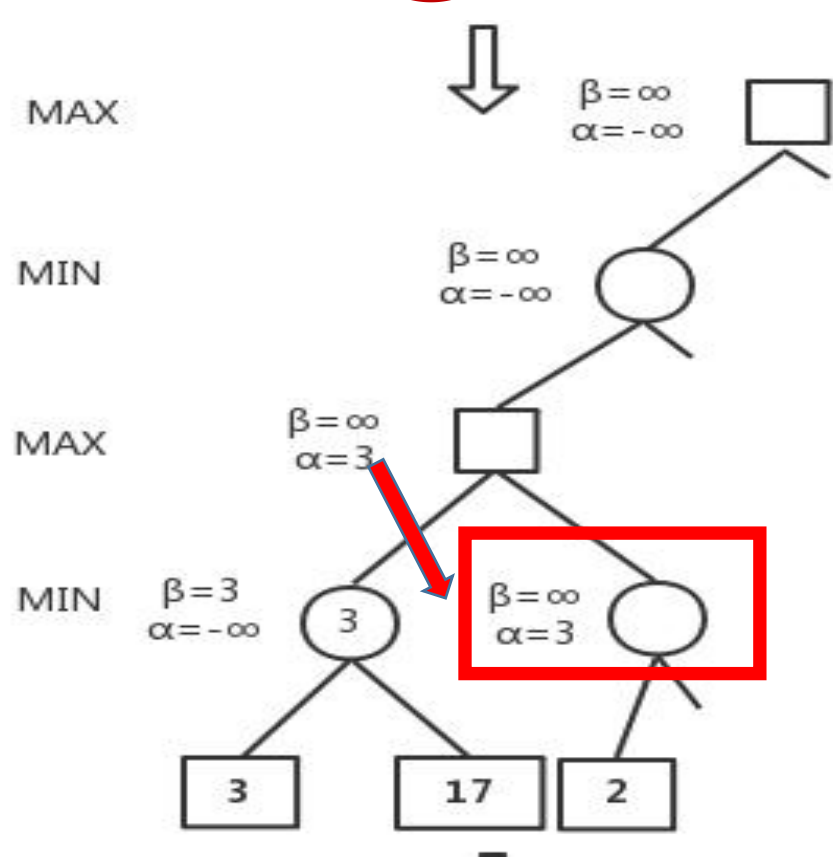
对抗搜索：Alpha-Beta 剪枝搜索示意

5



MAX节点, $\alpha = 3$

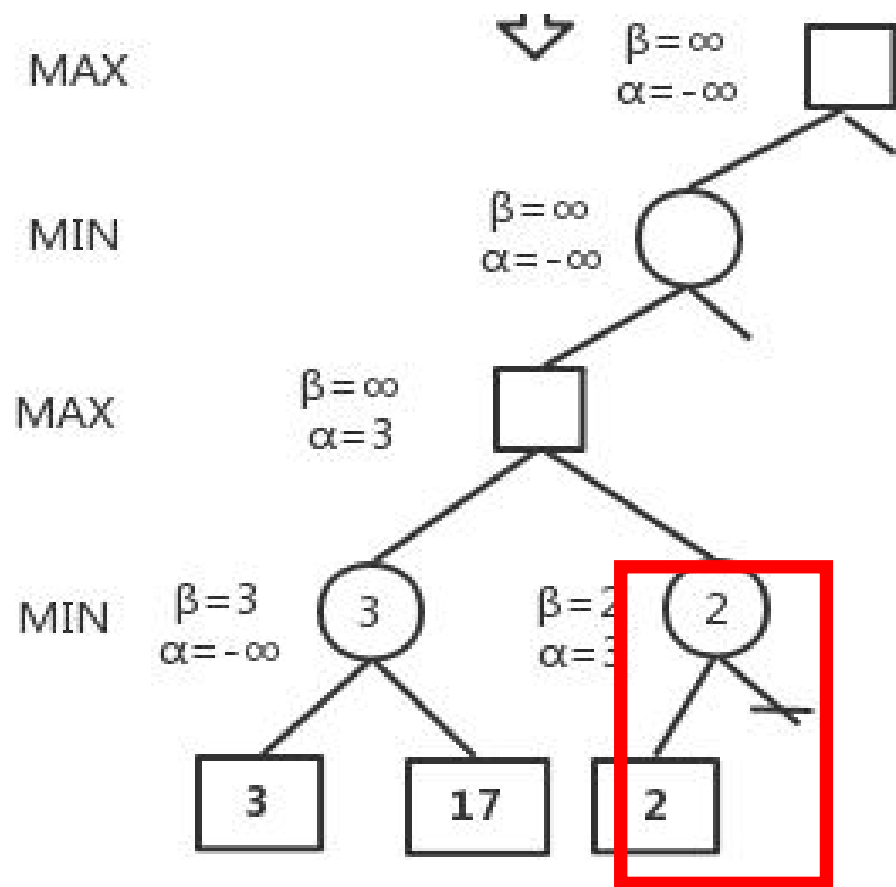
6



MIN节点, 先继承, α 为3

对抗搜索：Alpha-Beta 剪枝搜索示意

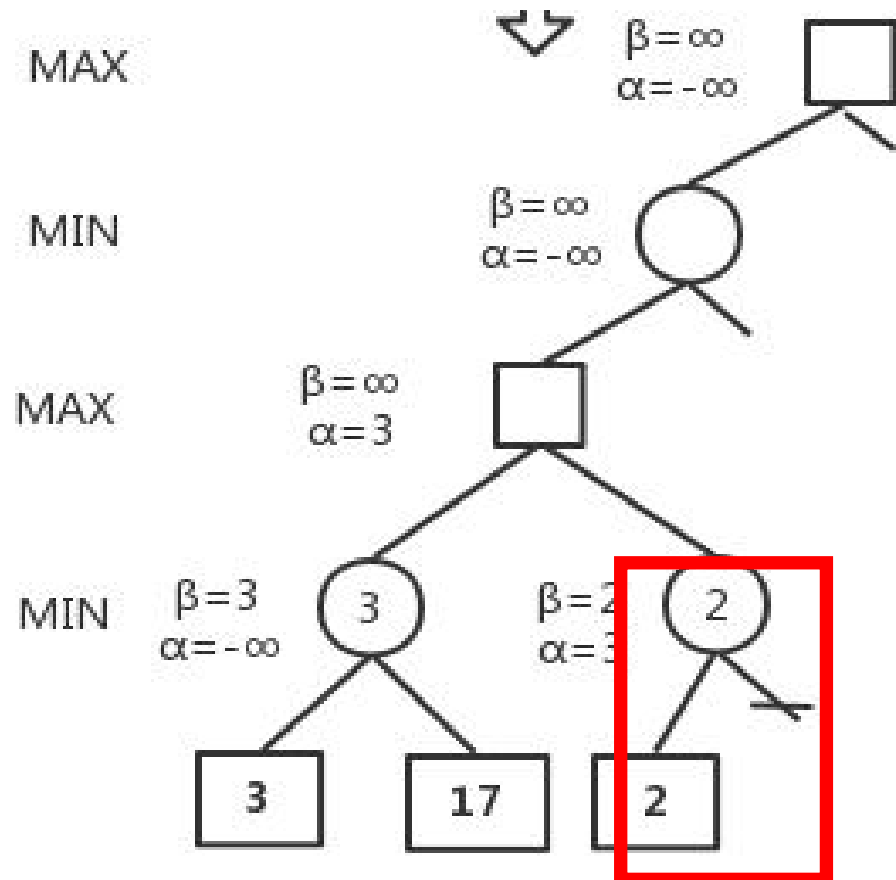
7



MIN 节点, $\beta = 2$, $\alpha > \beta$, 剪枝

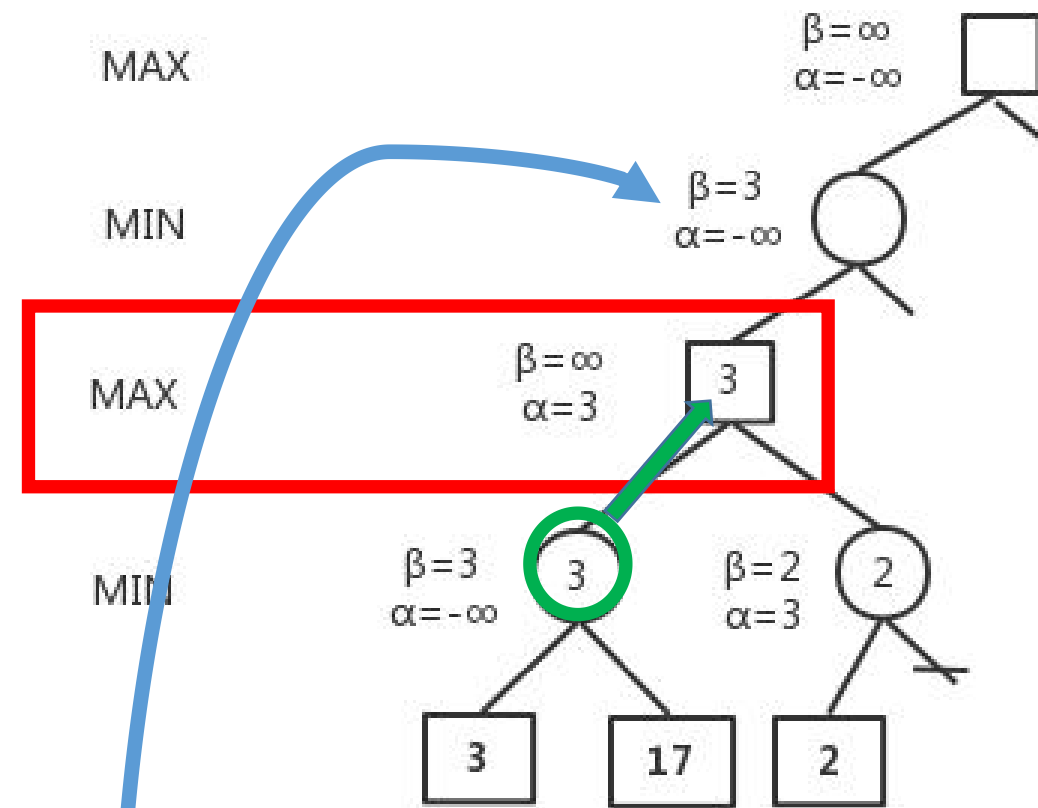
对抗搜索：Alpha-Beta 剪枝搜索示意

7



*MIN*节点, $\beta = 2$, $\alpha > \beta$, 剪枝

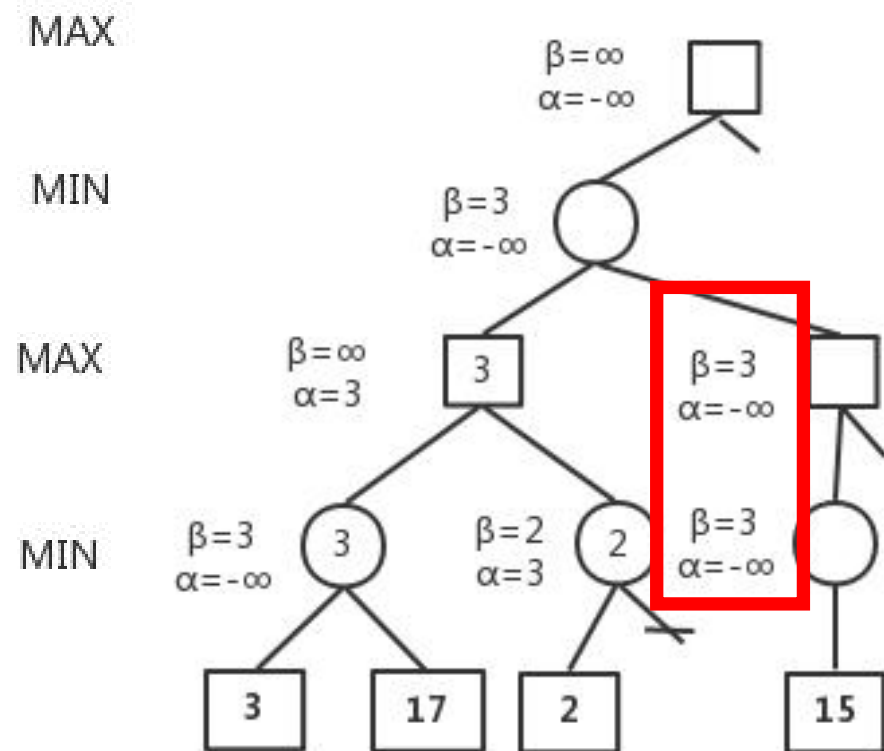
8



*MAX*节点, α 选最大: 3,
*MIN*节点, 更新 $\beta = 3$

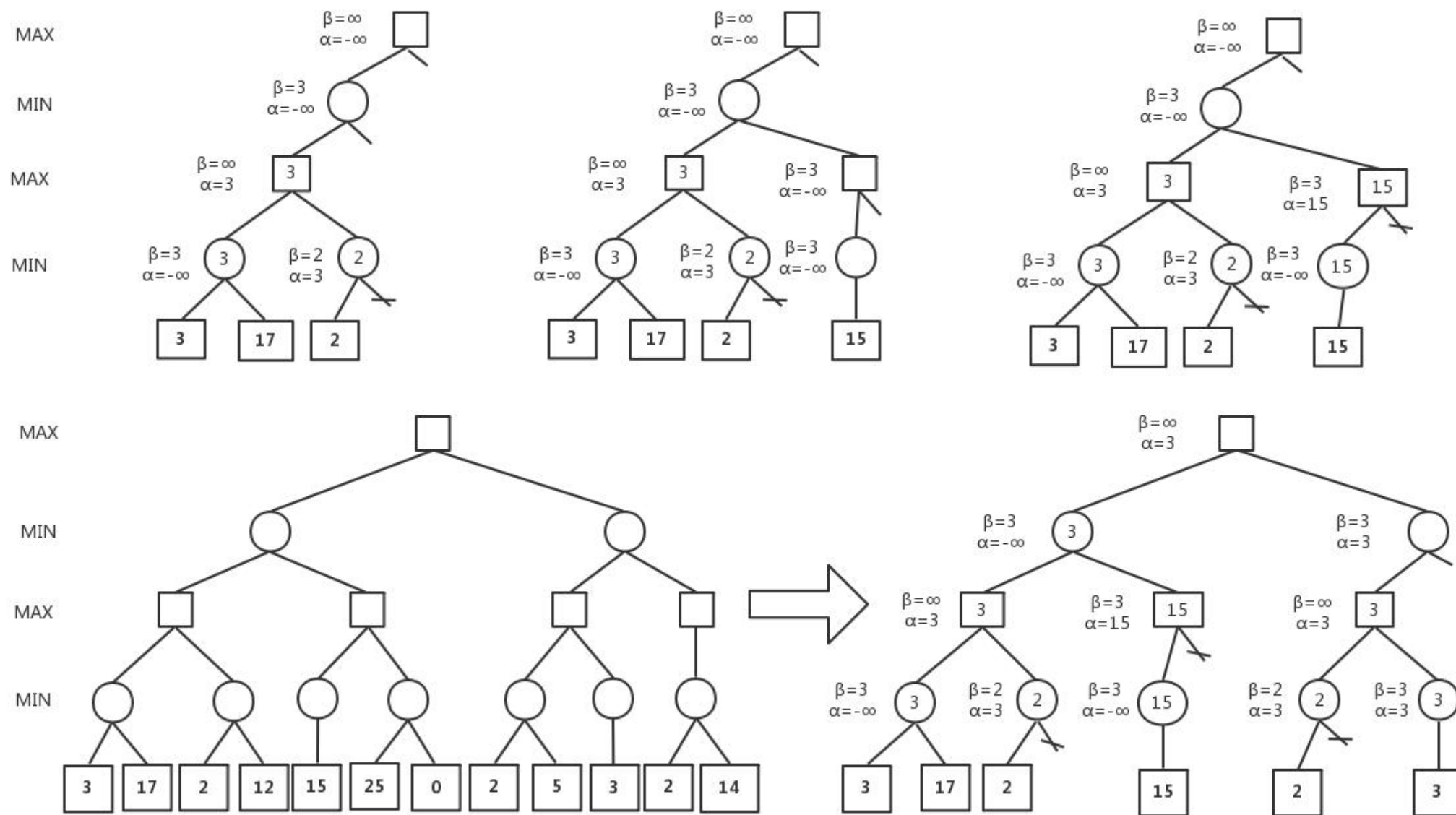
对抗搜索：Alpha-Beta 剪枝搜索示意

9



继承父节点,

对抗搜索：Alpha-Beta 剪枝搜索示意



对抗搜索：Alpha-Beta 剪枝搜索的性质

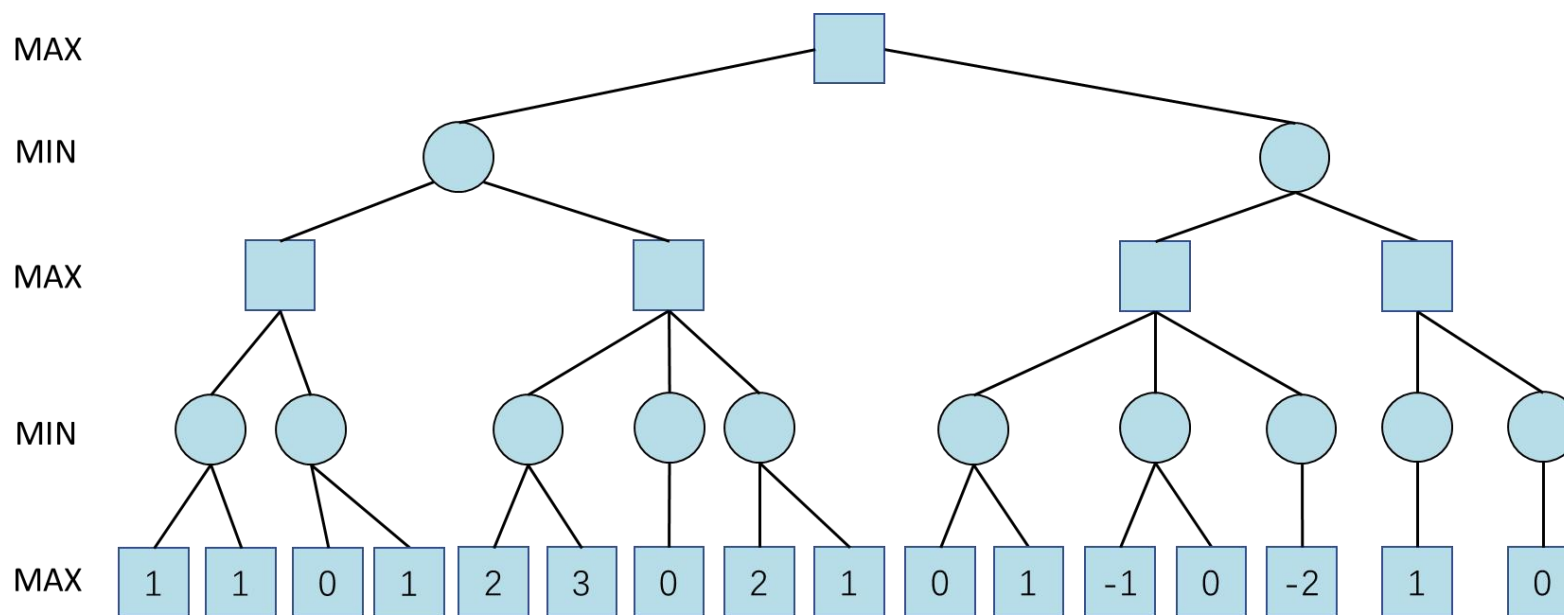
- 剪枝本身不影响算法输出结果
- 节点先后次序会影响剪枝效率
- 如果节点次序“恰到好处”，Alpha-Beta剪枝的时间复杂度为 $O(b^{\frac{m}{2}})$ ，最小最大搜索的时间复杂度为 $O(b^m)$

b	分支因子，即搜索树中每个节点最大的分支数目
m	搜索树中路径的最大可能长度

课上习题

图2展示了一棵Minimax搜索树，可采用alpha-beta剪枝算法进行对抗搜索。假设对于每个节点的后继节点，算法按照从左向右的方向扩展。同时假设当alpha值等于beta值时，算法不进行剪枝。请问：

- (1)对图2(a)中所示搜索树进行搜索，请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。



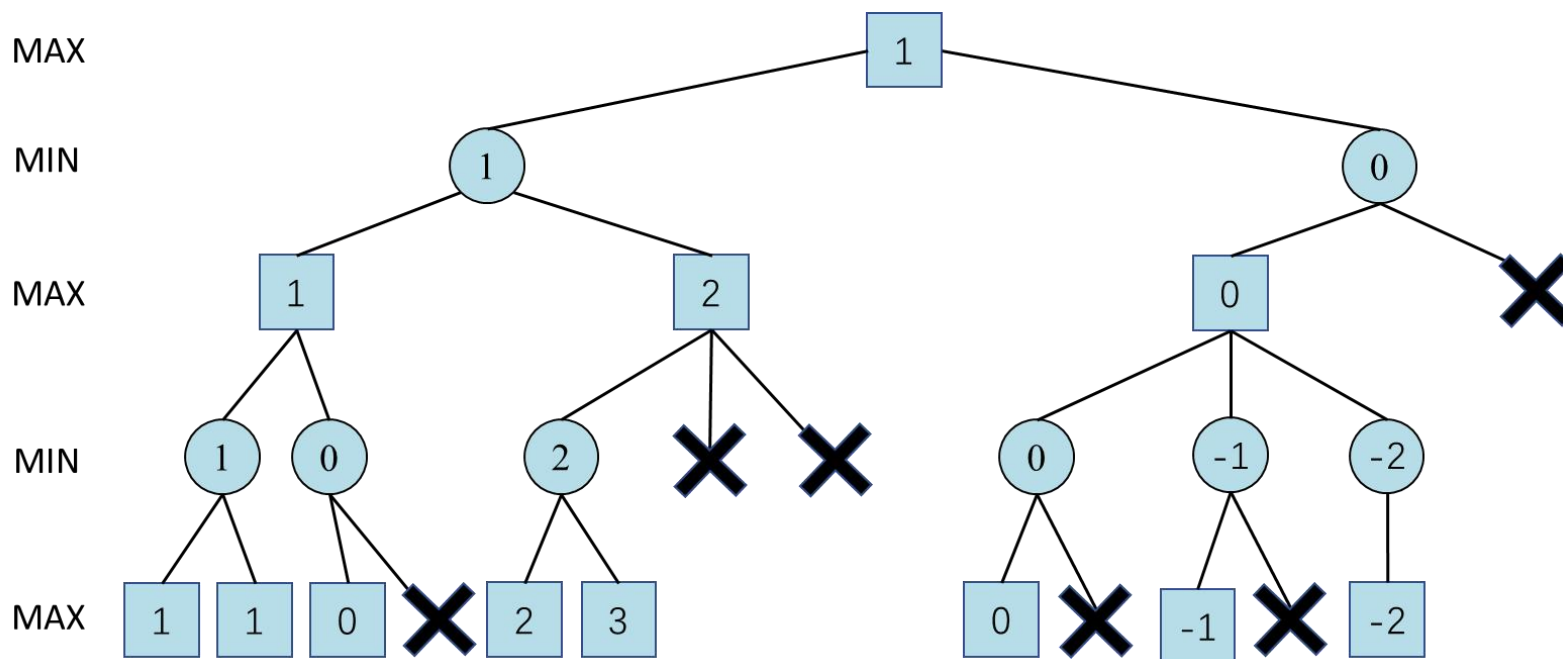
(a)

课上习题

图2展示了一棵Minimax搜索树，可采用alpha-beta剪枝算法进行对抗搜索。假设对于每个节点的后继节点，算法按照从左向右的方向扩展。同时假设当alpha值等于beta值时，算法不进行剪枝。请问：

- (1)对图2(a)中所示搜索树进行搜索，请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。

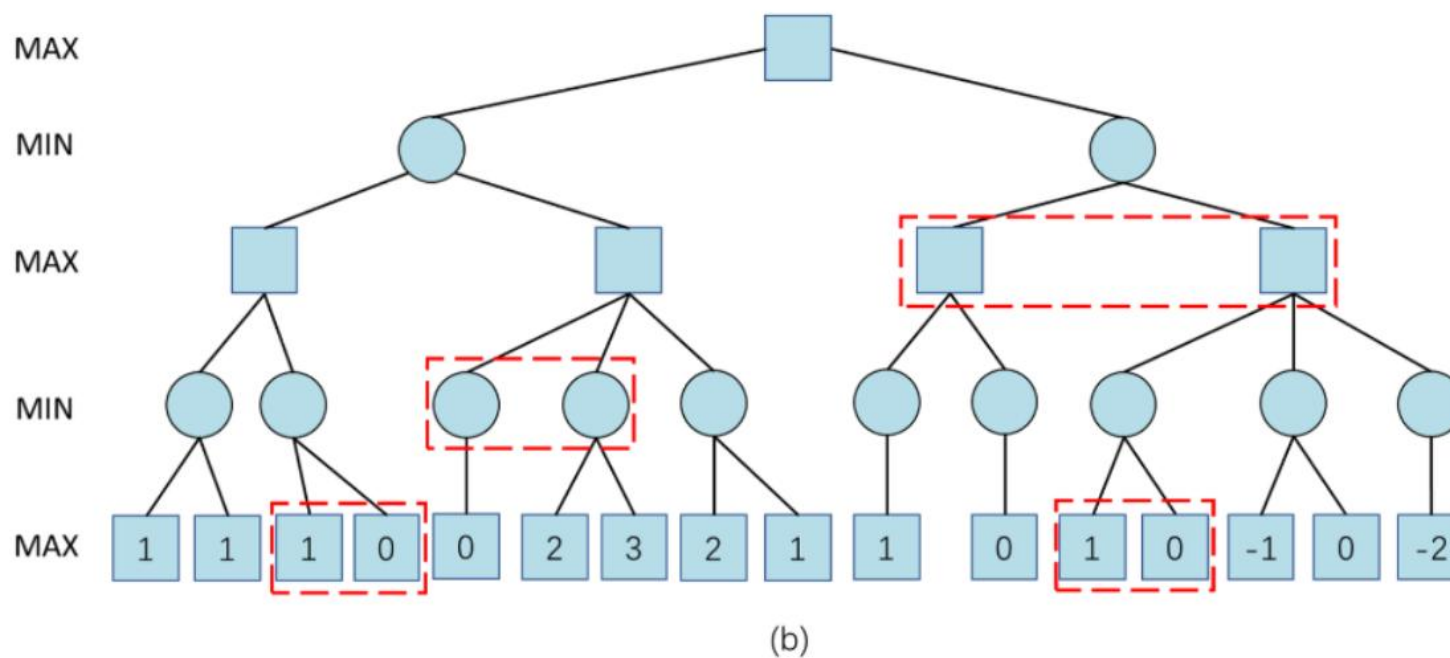
(1) 扩展节点数量为20。



课上习题

图2展示了一棵Minimax搜索树，可采用alpha-beta剪枝算法进行对抗搜索。假设对于每个节点的后继节点，算法按照从左向右的方向扩展。同时假设当alpha值等于beta值时，算法不进行剪枝。请问：

- (2)图2(b)展示了和图2(a)中完全相同的搜索树，只不过对搜索算法的顺序做了一些调整（扩展顺序交换的节点用红色虚线框标出）。请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。

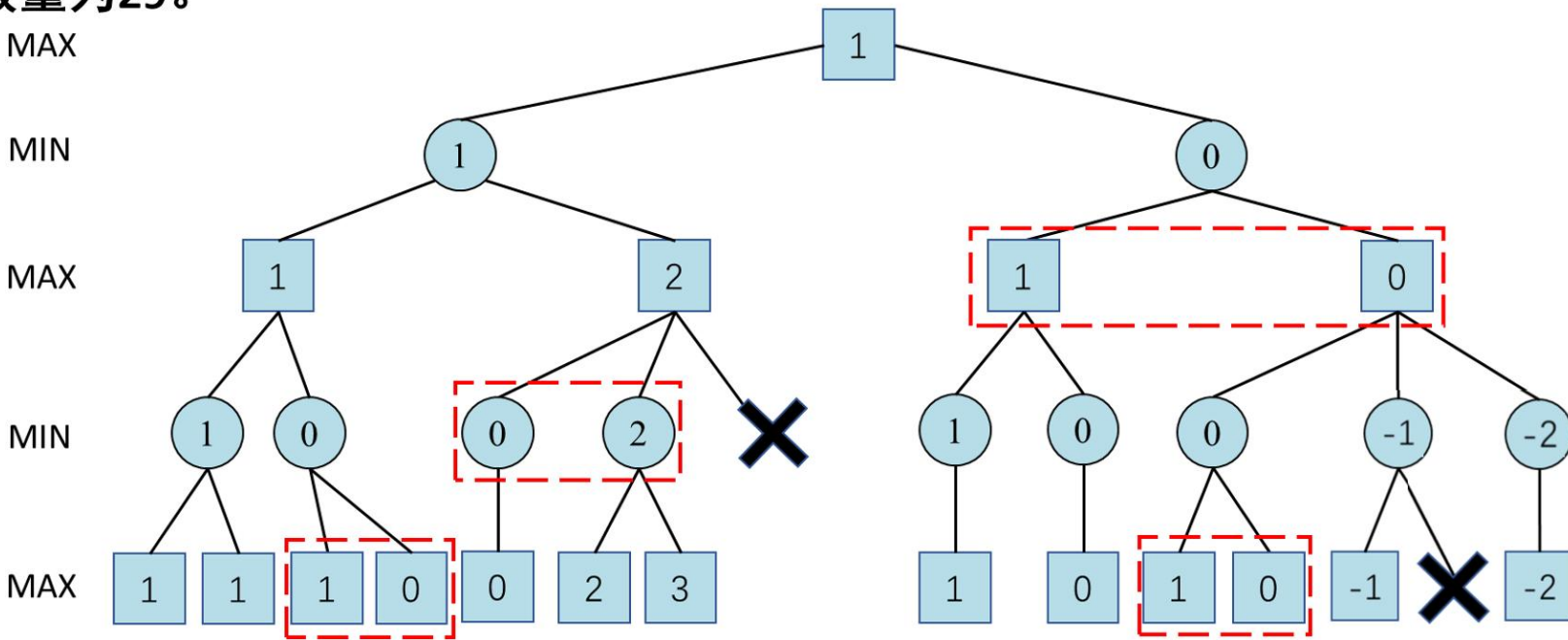


课上习题

图2展示了一棵Minimax搜索树，可采用alpha-beta剪枝算法进行对抗搜索。假设对于每个节点的后继节点，算法按照从左向右的方向扩展。同时假设当alpha值等于beta值时，算法不进行剪枝。请问：

- (2)图2(b)展示了和图2(a)中完全相同的搜索树，只不过对搜索算法的顺序做了一些调整（扩展顺序交换的节点用红色虚线框标出）。请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。

(2) 扩展节点数量为29。



提纲

- 搜索算法基础
- 启发式搜索
- 对抗搜索
- 蒙特卡洛树搜索

蒙特卡洛树搜索

• 推荐阅读材料

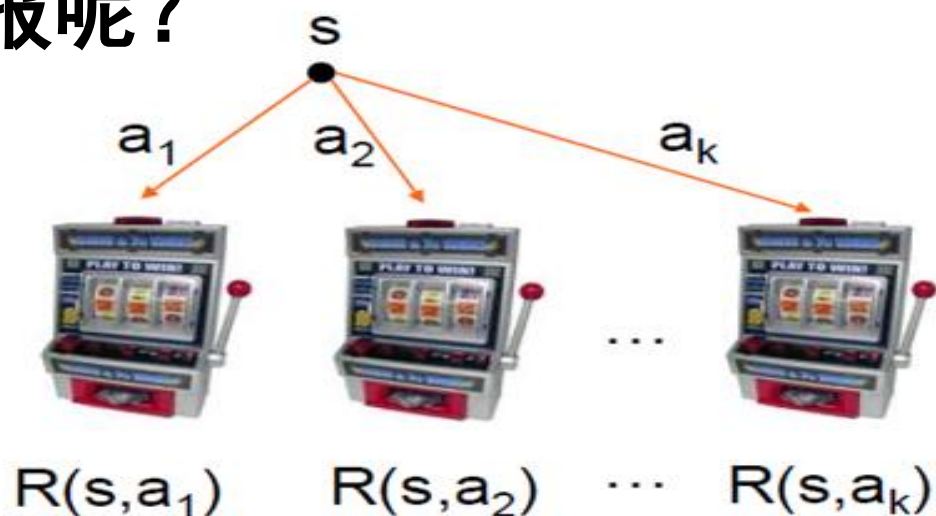
- D. Silver, et.al., Mastering the game of Go with Deep Neural Networks and Tree Search, [Nature](#), 529:484-490,2016
- C. Browne, et.al., Survey of Monte Carlo Tree Search Methods, [IEEE Transactions on Computational Intelligence and AI in Games](#), 4(1):1-49,2012
- S. Gelly, et al., The Grand Challenge of Computer Go: Monte Carlo Tree Search and Extensions, [Communications of the ACM](#), 55(3):106-113,2012
- L. Kocsis and C. Szepesvari, Bandit Based Monte-Carlo Planning, [ECML](#) 2006
- P. Auer, et. al., Finite-time analysis of the multi-armed bandit problem, [Machine learning](#), 47(2), 235-256, 2002

蒙特卡洛规划 (Monte-Carlo Planning)

- 单一状态蒙特卡洛规划： 多臂赌博机 (multi-armed bandits)
- 上限置信区间策略 (Upper Confidence Bound Strategies, UCB)
- 蒙特卡洛树搜索 (Monte-Carlo Tree Search)
 - UCT (Upper Confidence Bounds on Trees)

单一状态蒙特卡洛规划：多臂赌博机

- 单一状态， k 种行动(即有 k 个摇臂)
- 在摇臂赌博机问题中，每次以随机采样形式采取一种行动 a ，好比随机拉动第 k 个赌博机的臂膀，得到 $R(s, a_k)$ 的回报。
- 问题：下一次需要拉动那个赌博机的臂膀，才能获得最大回报呢？



多臂赌博机

- 多臂赌博机问题是一种序列决策问题，这种问题需要在利用(exploitation)和探索(exploration)之间保持平衡。
 - 利用(exploitation)：保证在过去决策中得到最佳回报
 - 探索(exploration)：寄希望在未来能够得到更大回报
- 如果有 k 个赌博机，这 k 个赌博机产生的操作序列为 $X_{i,1}, X_{i,2}, \dots$ ($i = 1, \dots, K$)。在时刻 $t = 1, 2, \dots$ ，选择第 I_t 个赌博机后，可得到奖赏 $X_{I_t,t}$ ，则在 n 次操作 $I_1, \dots, I_n$ 后，可如下定义悔值函数：

$$R_n = \max_{i=1,\dots,k} \sum_{t=1}^n X_{i,t} - \sum_{t=1}^n X_{I_t,t}$$

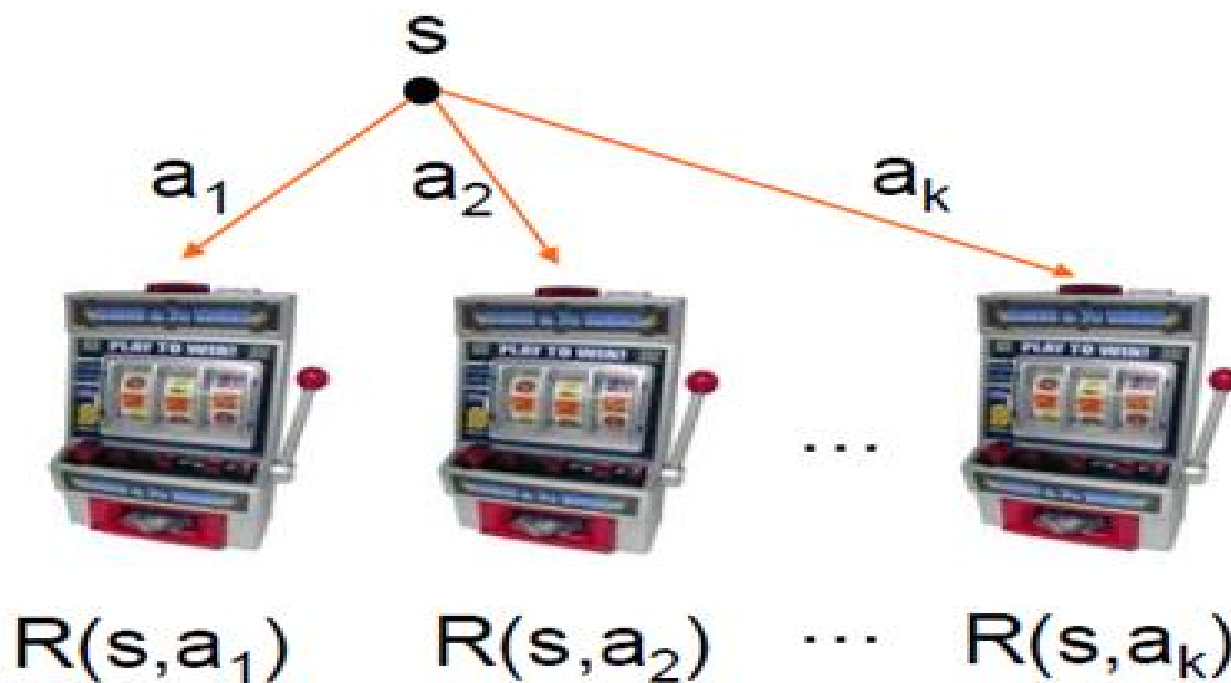
多臂赌博机

$$R_n = \max_{i=1,\dots,k} \sum_{t=1}^n X_{i,t} - \sum_{t=1}^n X_{I_{t,t}}$$

- 悔值函数表示了在第 t 次对赌博机操作时，假设知道哪个赌博机能够给出最大奖赏(虽然现实中不存在)，则将得到的最大奖赏减去实际操作第 I_t 个赌博机所得到的奖赏。将 n 次操作的差值累加起来，就是悔值函数的结果。
- 很显然，问题求解的目标为最小悔值函数的期望，智能体所采取的策略越优，收益越高，悔值函数值越小。

ϵ - 贪心算法

- 以 $1-\epsilon$ 的概率选择在过去 $t-1$ 次摇动赌博机臂膀行动中所获得平均收益分数最高的赌博机进行摇动；
- 以 ϵ 的概率随机选择一个赌博机进行摇动。



上限置信区间 (Upper Confidence Bound, UCB)

- 在多臂赌博机的研究过程中，上限置信区间成为一种较为成功的策略学习方法，因为其在探索-利用之间取得平衡。
- 在UCB方法中，使 $X_{i,T_i(t-1)}$ 来记录第 i 个赌博机在过去 $t - 1$ 时刻内的平均奖赏，则在第 t 时刻，选择使如下具有最佳上限置信区间的赌博机：

$$I_t = \max_{i \in \{1, \dots, k\}} \{ \overline{X_{i,T_i(t-1)}} + c_{t-1,T_i(t-1)} \}$$

其中 $c_{t,s}$ 取值定义如下： $c_{t,T_i(t-1)} = \sqrt{\frac{2 \ln n}{T_i(t-1)}}$

$T_i(t) = \sum_{s=1}^t \mathbb{I}(I_s = i)$ 为在过去时刻(初始时刻到 t 时刻)过程中选择第 i 个赌博机的次数总和。

上限置信区间 (Upper Confidence Bound, UCB)

- 也就是说，在第 t 时刻，UCB算法一般会选择具有如下最大值的第 j 个赌博机：

$$UCB = \overline{X}_j + \sqrt{\frac{2\ln n}{n_j}} \quad \text{或者} \quad UCB = \overline{X}_j + C \times \sqrt{\frac{2\ln n}{n_j}}$$

$\overline{X}_j$ 是第 j 个赌博机在过去时间内所获得的平均奖赏值， n_j 是在过去时间内拉动第 j 个赌博机臂膀的总次数， n 是过去时间内拉动所有赌博机臂膀的总次数。 C 是一个平衡因子，其决定着在选择时偏重探索还是利用。

上限置信区间 (Upper Confidence Bound, UCB)

- 也就是说，在第 t 时刻，UCB算法一般会选择具有如下最大值的第 j 个赌博机：

$$UCB = \overline{X}_j + \sqrt{\frac{2\ln n}{n_j}} \quad \text{或者} \quad UCB = \overline{X}_j + C \times \sqrt{\frac{2\ln n}{n_j}}$$

从这里可看出UCB算法如何在探索-利用之间寻找平衡：既需要拉动在过去时间内获得最大平均奖赏的赌博机，又希望去选择那些拉动臂膀次数最少的赌博机。

上限置信区间

- UCB算法描述

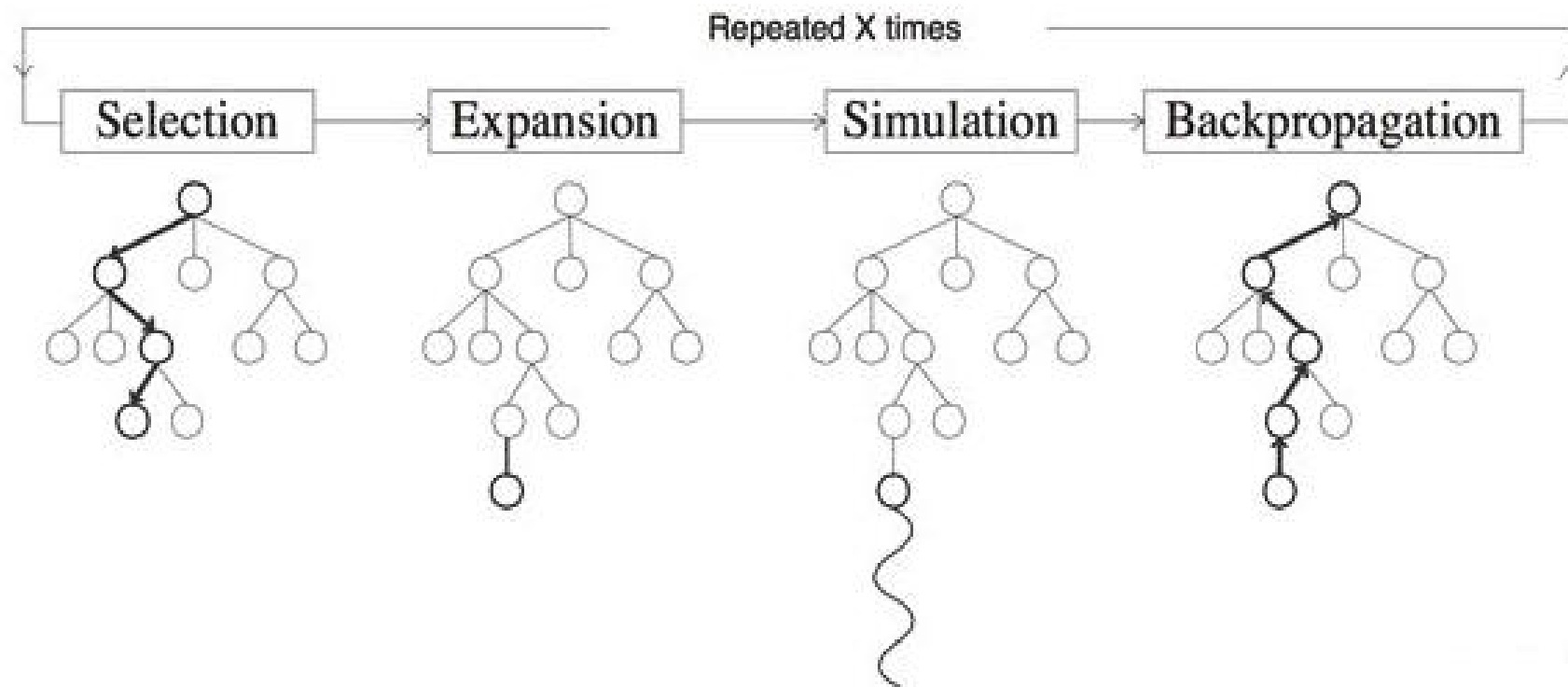
Deterministic policy: UCB1.

Initialization: Play each machine once.

Loop: Play machine j that maximizes $\bar{x}_j + \sqrt{\frac{2\ln n}{n_j}}$, where $\bar{x}_j$ is the average reward obtained from machine j , n_j is the number of times machine j has been explored so far, and n is the overall number of plays done so far.

蒙特卡洛树搜索(Monte-Carlo Tree Search, MCTS)

- 将上限置信区间算法UCB应用于游戏树的搜索方法
 - 包括了四个步骤：选择(selection)，扩展(expansion)，模拟(simulation)，反向传播(Back-Propagation)

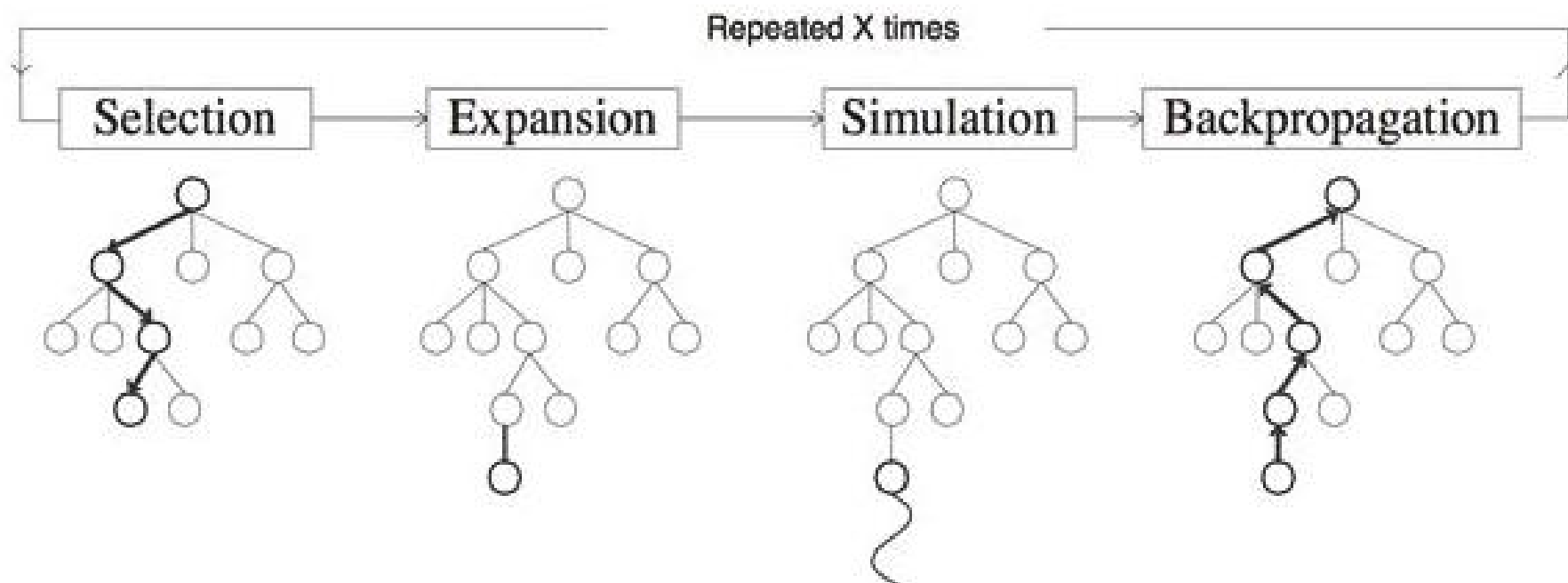


L. Kocsis and C. Szepesvari, Bandit based Monte-Carlo Planning, [ECML](#), 2006:282–293

蒙特卡洛树搜索

- **选择：** 从根节点 R 开始，递归选择子节点，直至到达叶节点或到达具有还未被扩展过的子节点的节点 L。
- 具体来说，通常用UCB1 (Upper Confidence Bound, 上限置信区间)选择最具“潜力”的后续节点

$$UCB = \overline{X}_j + \sqrt{\frac{2 \ln n}{n_j}}$$



蒙特卡洛树搜索

- **扩展：**

- 如果 L 不是一个终止节点，则随机创建其后的一个未被访问节点，选择该节点作为后续子节点 C 。

- **模拟：**

- 从节点 C 出发，对游戏进行模拟，直到博弈游戏结束。

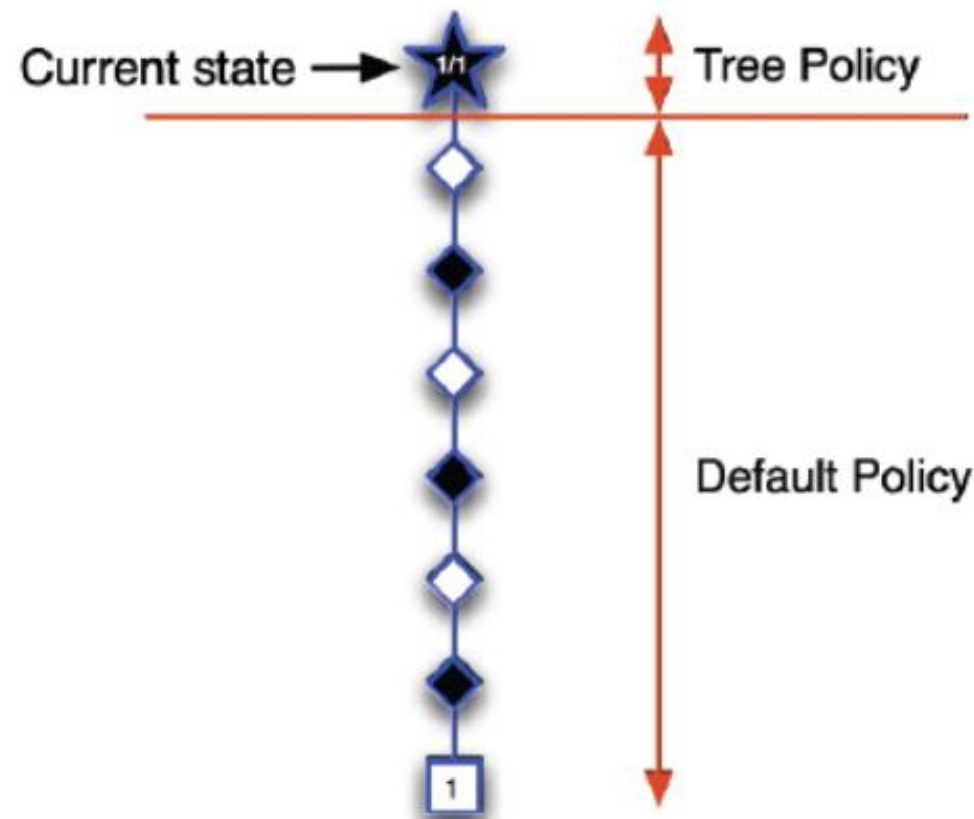
- **反向传播**

- 用模拟所得结果来回溯更新导致这个结果的每个节点中获胜次数和访问次数。

蒙特卡洛树搜索

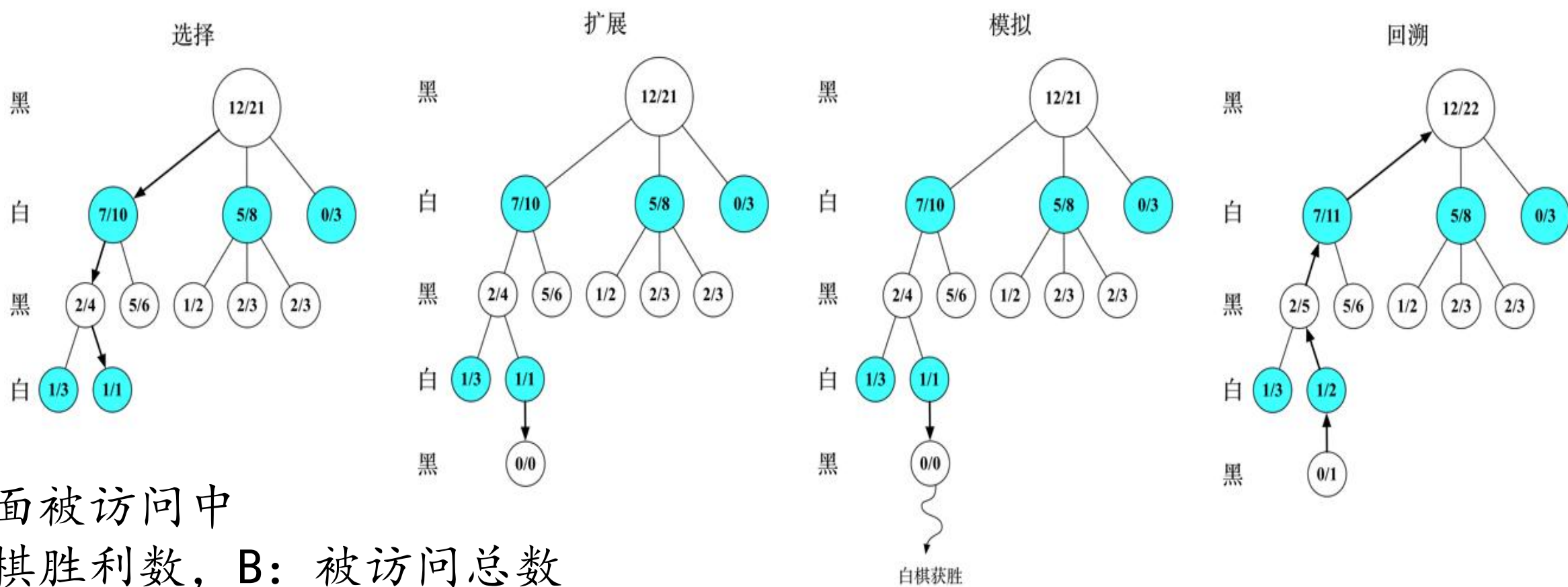
- 两种策略学习机制：

- **搜索树策略**：从已有的搜索树中选择或创建一个叶子结点(即**选择**和**拓展**)。搜索树策略需要在利用和探索之间保持平衡。
- **模拟策略**：从非叶子结点出发模拟游戏，得到游戏仿真结果。



蒙特卡洛树搜索：例子

- 以围棋为例，假设根节点是执黑棋方。
- 图中每一个节点都代表一个局面，每一个局面记录两个值A/B：



蒙特卡洛树搜索：例子

- 假设由黑棋行棋，取一个UCB1值最大的节点作为后续节点：

左1：7/10对应

$$\frac{7}{10} + \sqrt{\frac{\ln(21)}{10}} = 1.252$$

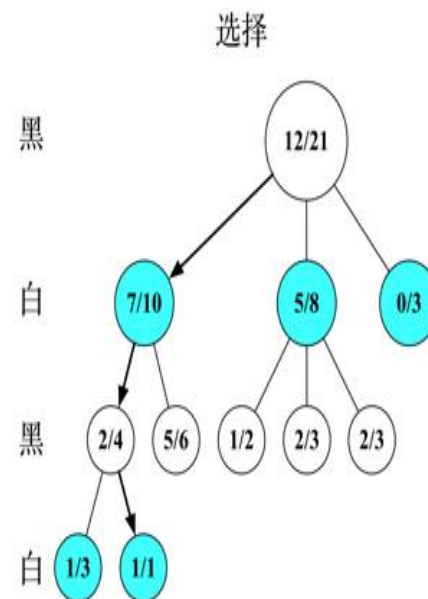
左2，5/8对应

$$\frac{5}{8} + \sqrt{\frac{\ln(21)}{8}} = 1.243$$

左3，0/3对应

$$\frac{0}{3} + \sqrt{\frac{\ln(21)}{3}} = 1.007$$

黑棋会选择局面7/10



蒙特卡洛树搜索：例子

- 在节点7/10，由**白棋**行棋(第一项为1-A/B)，由UCB1公式可得

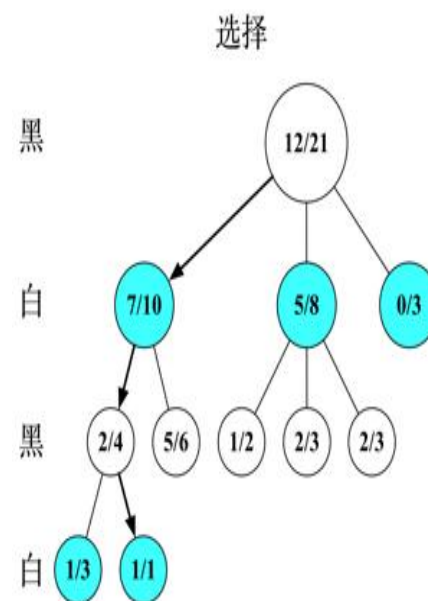
左1，2/4对应

$$\frac{2}{4} + \sqrt{\frac{\ln(10)}{4}} = 1.26$$

左2，5/6对应

$$\frac{1}{6} + \sqrt{\frac{\ln(10)}{6}} = 0.786$$

白棋会选择局面2/4



蒙特卡洛树搜索：例子

- 在节点2/4，黑棋评估下面的两个局面，由UCB1公式可得

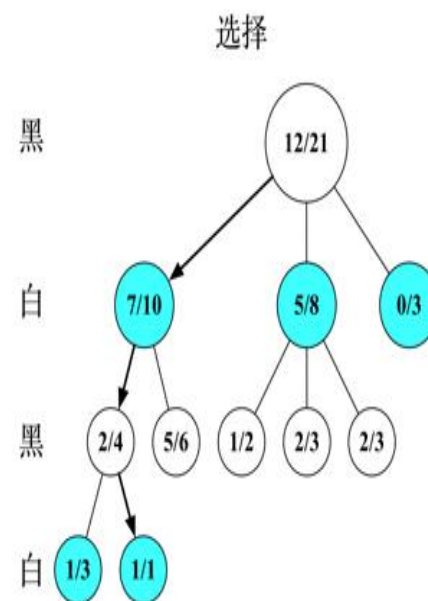
左1，1/3对应

$$\frac{1}{3} + \sqrt{\frac{\ln(4)}{3}} = 1.01$$

左1，1/1对应

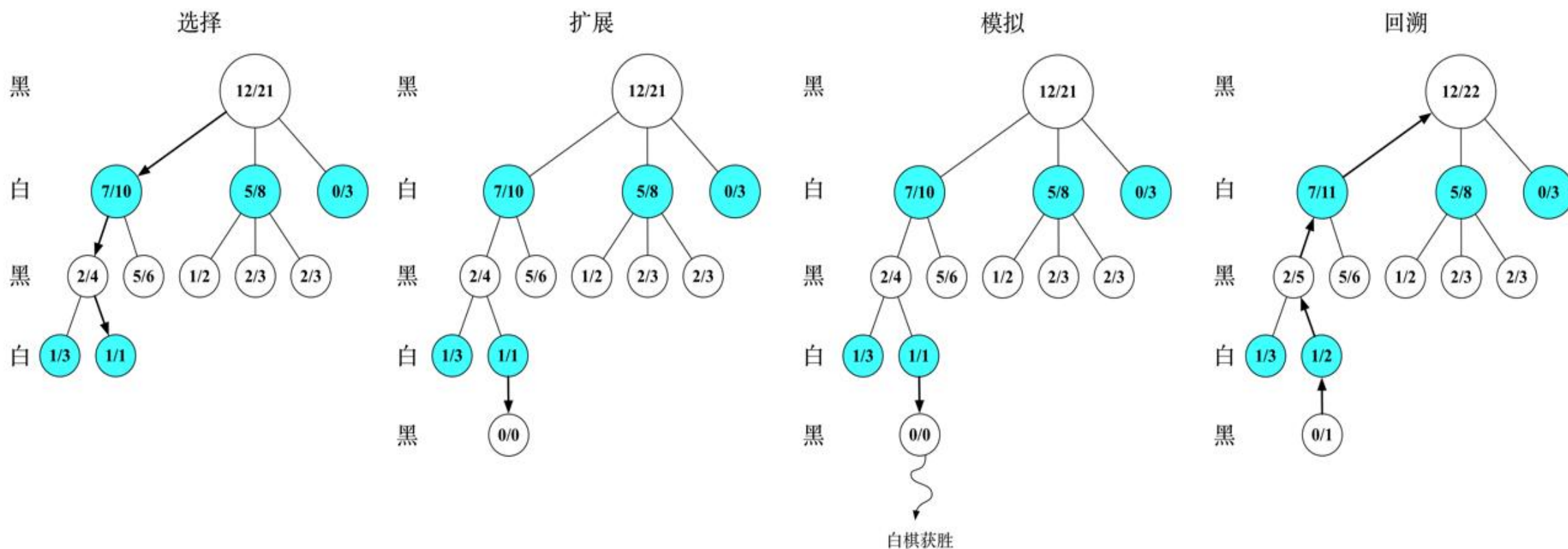
$$\frac{1}{1} + \sqrt{\frac{\ln(4)}{1}} = 2.18$$

黑棋会选择局面1/1。已达叶结点，需要扩展



蒙特卡洛树搜索：例子

- 随机扩展一个新节点，初始化为0/0，在该节点下进行模拟
- 假设经过一系列仿真行棋后，最终白棋获胜。该新节点的A/B值被更新为0/1，并向上回溯，所有父辈节点的A不变，B加1



使用蒙特卡洛树搜索的原因

- 蒙特卡洛树搜索基于**采样**来得到结果、而非**穷尽式枚举** (虽然在枚举过程中也可剪掉若干不影响结果的分支)。

蒙特卡洛树搜索算法

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f: S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

蒙特卡洛树搜索算法(UCT)

function UCTSEARCH(s_0)

→ create root node v_0 with state s_0
while within computational budget **do**
 → $v_l \leftarrow \text{TREEPOLICY}(v_0)$
 $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$
 BACKUP(v_l, Δ)
return $a(\text{BESTCHILD}(v_0, 0))$

function TREEPOLICY(v)

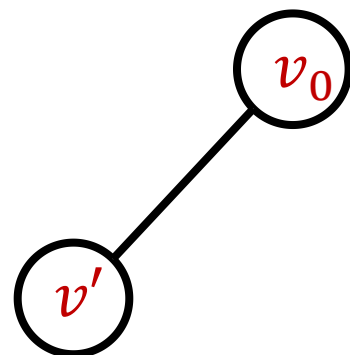
while v is nonterminal **do**
 if v not fully expanded **then**
 → **return** EXPAND(v)
else
 $v \leftarrow \text{BESTCHILD}(v, C_p)$
return v

function DEFAULTPOLICY(s)

while s is non-terminal **do**
 choose $a \in A(s)$ uniformly at random
 $s \leftarrow f(s, a)$
return reward for state s

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$



function BACKUP(v, Δ)

while v is not null **do**
 $N(v) \leftarrow N(v) + 1$
 $Q(v) \leftarrow Q(v) + \Delta(v, p)$
 $v \leftarrow \text{parent of } v$

function EXPAND(v)

choose $a \in \text{untried actions from } A(s(v))$
 → add a new child v' to v
 with $s(v') = f(s(v), a)$
 and $a(v') = a$
return v'

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

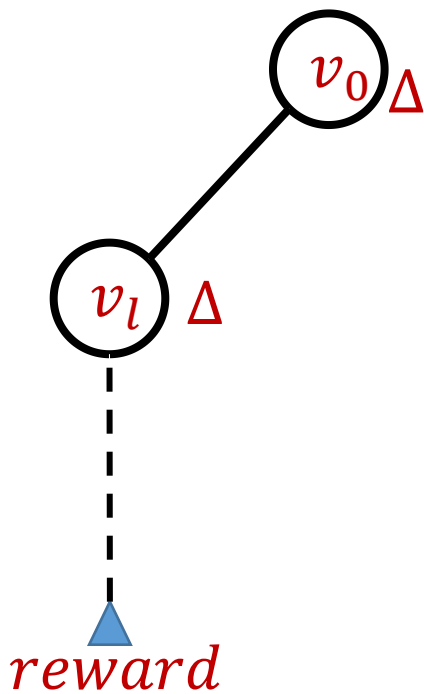
蒙特卡洛树搜索算法(UCT)

```
function UCTSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
     $\text{BACKUP}(v_l, \Delta)$   
  return  $a(\text{BESTCHILD}(v_0, 0))$ 
```

```
function TREEPOLICY( $v$ )  
  while  $v$  is nonterminal do  
    if  $v$  not fully expanded then  
      return  $\text{EXPAND}(v)$   
    else  
       $v \leftarrow \text{BESTCHILD}(v, C_p)$   
  return  $v$ 
```

```
function DEFAULTPOLICY( $s$ )  
  while  $s$  is non-terminal do  
    choose  $a \in A(s)$  uniformly at random  
     $s \leftarrow f(s, a)$   
  return reward for state  $s$ 
```

```
function BESTCHILD( $v, c$ )  
  
  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
```



```
function BACKUP( $v, \Delta$ )  
  while  $v$  is not null do  
     $N(v) \leftarrow N(v) + 1$   
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$   
     $v \leftarrow \text{parent of } v$ 
```

```
function EXPAND( $v$ )  
  choose  $a \in \text{untried actions from } A(s(v))$   
  add a new child  $v'$  to  $v$   
    with  $s(v') = f(s(v), a)$   
    and  $a(v') = a$   
  return  $v'$ 
```

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

蒙特卡洛树搜索算法(UCT)

function UCTSEARCH(s_0)

create root node v_0 with state s_0

→ **while** within computational budget **do**

→ $v_l \leftarrow \text{TREEPOLICY}(v_0)$

$\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$

BACKUP(v_l, Δ)

return $a(\text{BESTCHILD}(v_0, 0))$

function TREEPOLICY(v)

while v is nonterminal **do**

if v not fully expanded **then**

return EXPAND(v)

else

→ $v \leftarrow \text{BESTCHILD}(v, C_p)$

return v

function DEFAULTPOLICY(s)

while s is non-terminal **do**

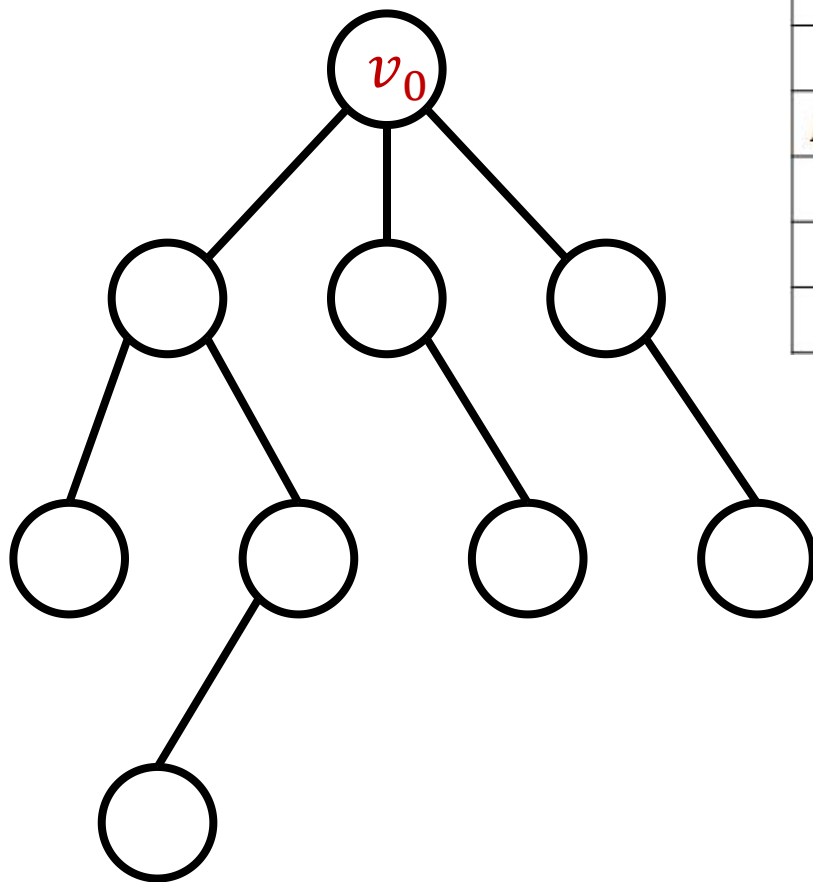
choose $a \in A(s)$ uniformly at random

$s \leftarrow f(s, a)$

return reward for state s

function BESTCHILD(v, c)

return $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$



S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

function BACKUP(v, Δ)

while v is not null **do**

$N(v) \leftarrow N(v) + 1$

$Q(v) \leftarrow Q(v) + \Delta(v, p)$

$v \leftarrow \text{parent of } v$

function EXPAND(v)

choose $a \in \text{untried actions from } A(s(v))$

add a new child v' to v

with $s(v') = f(s(v), a)$

and $a(v') = a$

return v'

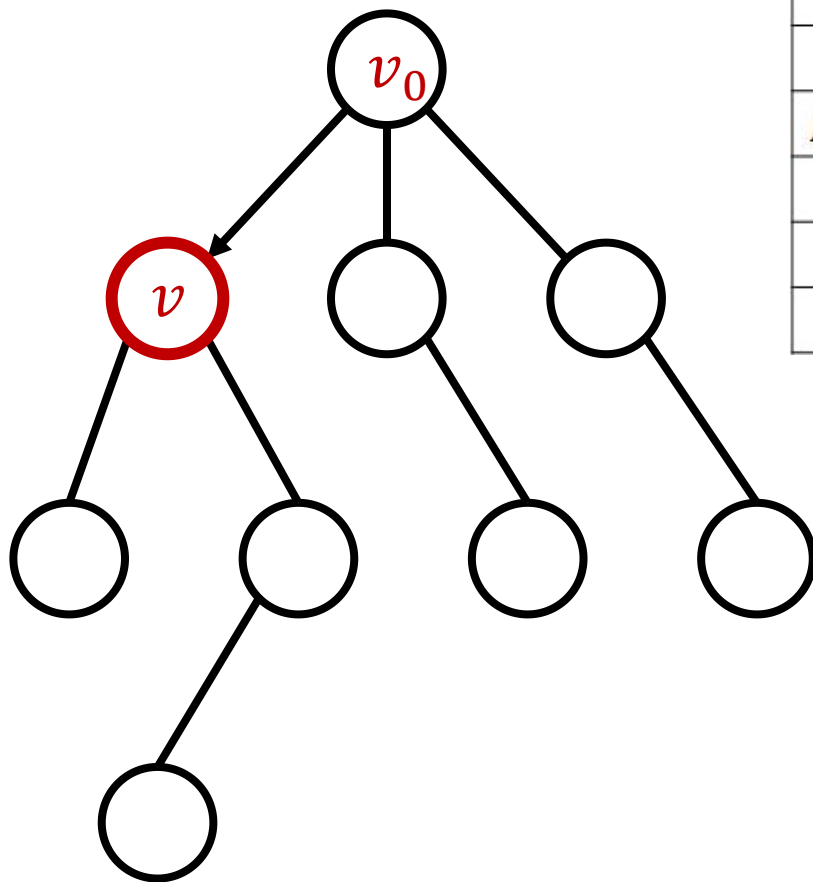
蒙特卡洛树搜索算法(UCT)

```
function UCTSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
     $\text{BACKUP}(v_l, \Delta)$   
  return  $a(\text{BESTCHILD}(v_0, 0))$ 
```

```
function TREEPOLICY( $v$ )  
  while  $v$  is nonterminal do  
    if  $v$  not fully expanded then  
      return  $\text{EXPAND}(v)$   
    else  
       $v \leftarrow \text{BESTCHILD}(v, C_p)$   
  return  $v$ 
```

```
function DEFAULTPOLICY( $s$ )  
  while  $s$  is non-terminal do  
    choose  $a \in A(s)$  uniformly at random  
     $s \leftarrow f(s, a)$   
  return reward for state  $s$ 
```

```
function BESTCHILD( $v, c$ )  
  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
```



S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

```
function BACKUP( $v, \Delta$ )  
  while  $v$  is not null do  
     $N(v) \leftarrow N(v) + 1$   
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$   
     $v \leftarrow \text{parent of } v$ 
```

```
function EXPAND( $v$ )  
  choose  $a \in \text{untried actions from } A(s(v))$   
  add a new child  $v'$  to  $v$   
    with  $s(v') = f(s(v), a)$   
    and  $a(v') = a$   
  return  $v'$ 
```

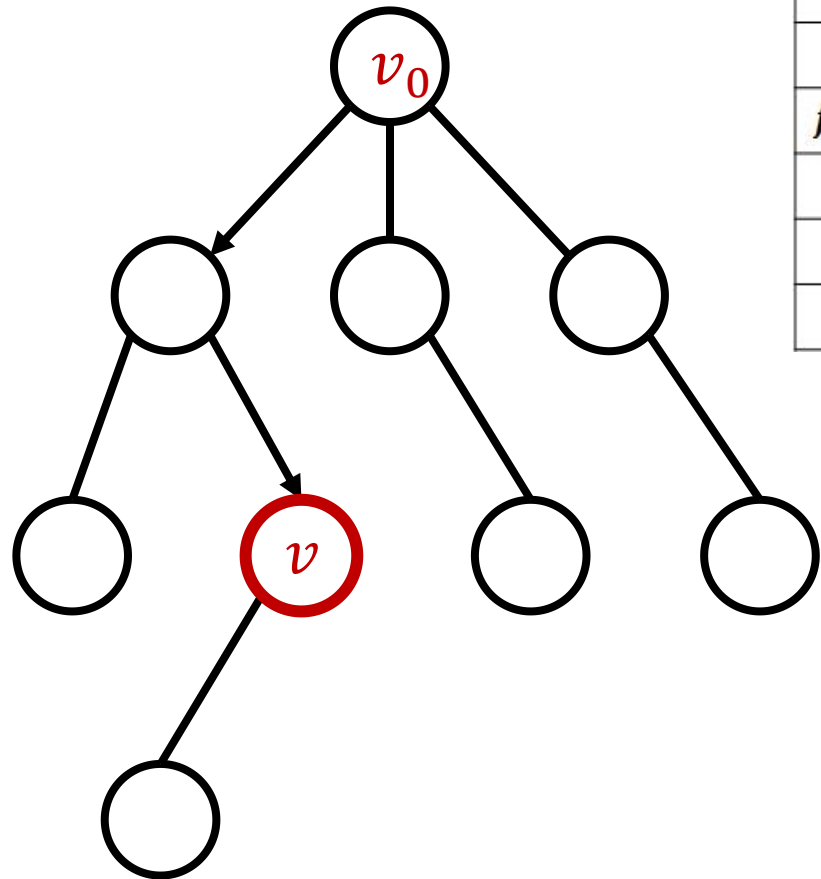
蒙特卡洛树搜索算法(UCT)

```
function UCTSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
     $\text{BACKUP}(v_l, \Delta)$   
  return  $a(\text{BESTCHILD}(v_0, 0))$ 
```

```
function TREEPOLICY( $v$ )  
→ while  $v$  is nonterminal do  
  if  $v$  not fully expanded then  
    return  $\text{EXPAND}(v)$   
  else  
     $v \leftarrow \text{BESTCHILD}(v, C_p)$   
  return  $v$ 
```

```
function DEFAULTPOLICY( $s$ )  
  while  $s$  is non-terminal do  
    choose  $a \in A(s)$  uniformly at random  
     $s \leftarrow f(s, a)$   
  return reward for state  $s$ 
```

```
function BESTCHILD( $v, c$ )  
  
  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
```



```
function  $\text{BACKUP}(v, \Delta)$   
  while  $v$  is not null do  
     $N(v) \leftarrow N(v) + 1$   
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$   
     $v \leftarrow \text{parent of } v$ 
```

```
function  $\text{EXPAND}(v)$   
  choose  $a \in \text{untried actions from } A(s(v))$   
  add a new child  $v'$  to  $v$   
    with  $s(v') = f(s(v), a)$   
    and  $a(v') = a$   
  return  $v'$ 
```

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

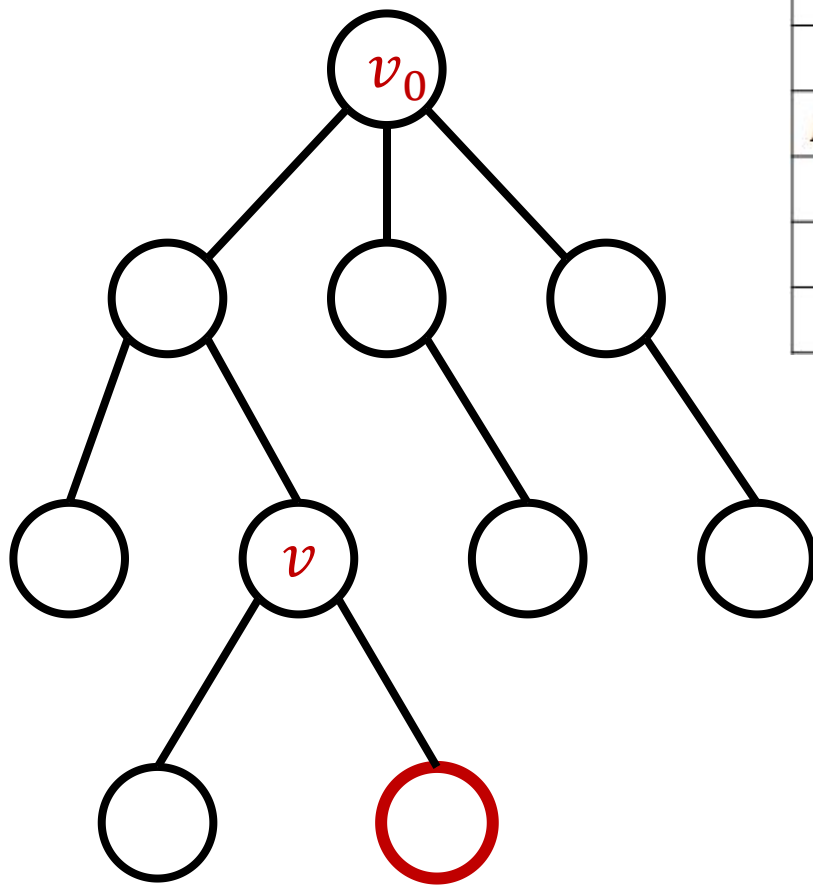
蒙特卡洛树搜索算法(UCT)

```
function UCTSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
    BACKUP( $v_l, \Delta$ )  
  return  $a(\text{BESTCHILD}(v_0, 0))$ 
```

```
function TREEPOLICY( $v$ )  
  while  $v$  is nonterminal do  
    if  $v$  not fully expanded then  
      return EXPAND( $v$ )  
    else  
       $v \leftarrow \text{BESTCHILD}(v, C_p)$   
  return  $v$ 
```

```
function DEFAULTPOLICY( $s$ )  
  while  $s$  is non-terminal do  
    choose  $a \in A(s)$  uniformly at random  
     $s \leftarrow f(s, a)$   
  return reward for state  $s$ 
```

```
function BESTCHILD( $v, c$ )  
  
  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v' )}}$ 
```



```
function BACKUP( $v, \Delta$ )  
  while  $v$  is not null do  
     $N(v) \leftarrow N(v) + 1$   
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$   
     $v \leftarrow \text{parent of } v$ 
```

```
function EXPAND( $v$ )  
  choose  $a \in \text{untried actions from } A(s(v))$   
  add a new child  $v'$  to  $v$   
    with  $s(v') = f(s(v), a)$   
    and  $a(v') = a$   
  return  $v'$ 
```

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(V)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

蒙特卡洛树搜索算法(UCT)

```

function UCTSEARCH( $s_0$ )
  create root node  $v_0$  with state  $s_0$ 
  while within computational budget do
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$ 
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$ 
     $\text{BACKUP}(v_l, \Delta)$ 
  return  $a(\text{BESTCHILD}(v_0, 0))$ 
    
```

```

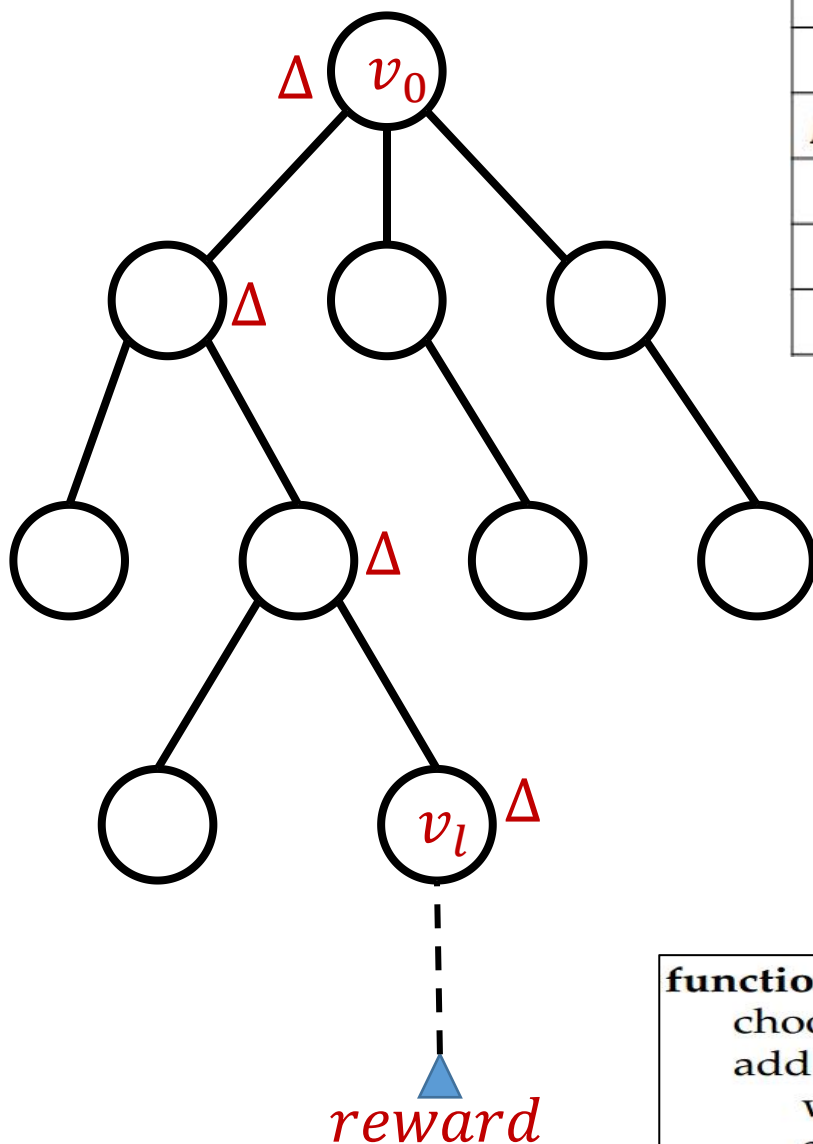
function TREEPOLICY( $v$ )
  while  $v$  is nonterminal do
    if  $v$  not fully expanded then
      return  $\text{EXPAND}(v)$ 
    else
       $v \leftarrow \text{BESTCHILD}(v, C_p)$ 
  return  $v$ 
    
```

```

function DEFAULTPOLICY( $s$ )
  while  $s$  is non-terminal do
    choose  $a \in A(s)$  uniformly at random
     $s \leftarrow f(s, a)$ 
  return reward for state  $s$ 
    
```

```

function BESTCHILD( $v, c$ )
  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
```



S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(v)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值

```

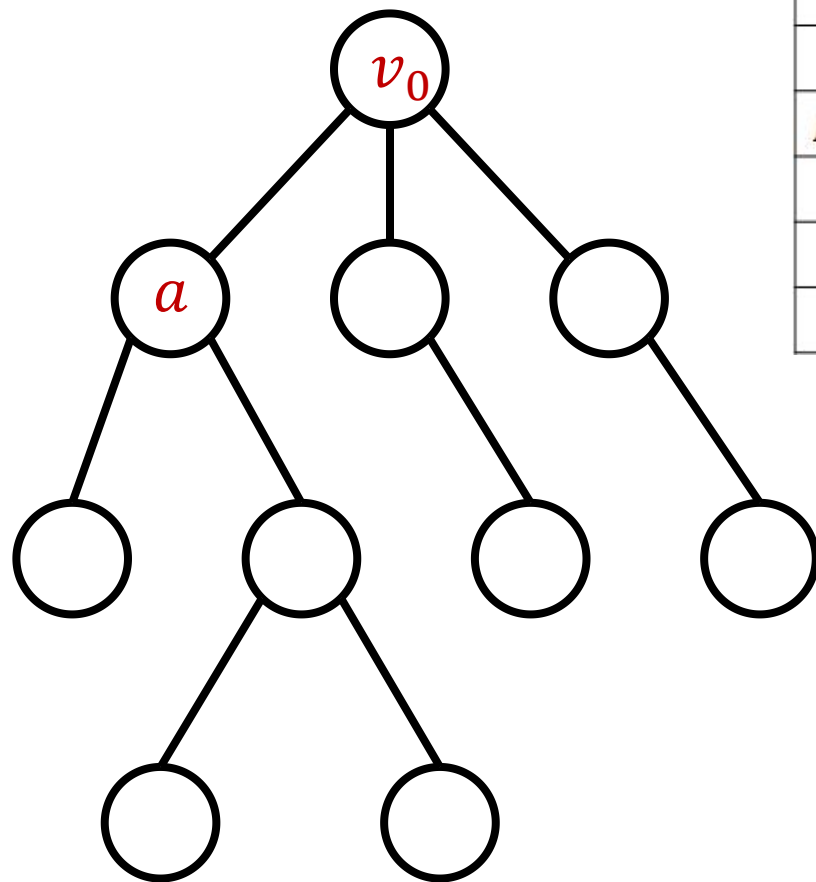
function BACKUP( $v, \Delta$ )
  while  $v$  is not null do
     $N(v) \leftarrow N(v) + 1$ 
     $Q(v) \leftarrow Q(v) + \Delta(v, p)$ 
     $v \leftarrow \text{parent of } v$ 
    
```

```

function EXPAND( $v$ )
  choose  $a \in$  untried actions from  $A(s(v))$ 
  add a new child  $v'$  to  $v$ 
    with  $s(v') = f(s(v), a)$ 
    and  $a(v') = a$ 
  return  $v'$ 
    
```


蒙特卡洛树搜索算法(UCT)

S	状态集
$A(s)$	在状态 s 能够采取的有效行动的集合
$s(v)$	节点 v 所代表的状态
$a(v)$	所采取的行动导致到达节点 v
$f : S \times A \rightarrow S$	状态转移函数
$N(v)$	节点 v 被访问的次数
$Q(v)$	节点 v 所获得的奖赏值
$\Delta(v, p)$	玩家 p 选择节点 v 所得到的奖赏值



```

function UCTSEARCH( $s_0$ )
    create root node  $v_0$  with state  $s_0$ 
    while within computational budget do
         $v_l \leftarrow \text{TREEPOLICY}(v_0)$ 
         $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$ 
         $\text{BACKUP}(v_l, \Delta)$ 
    return  $a(\text{BESTCHILD}(v_0, 0))$ 
    
```

```

function TREEPOLICY( $v$ )
    while  $v$  is nonterminal do
        if  $v$  not fully expanded then
            return  $\text{EXPAND}(v)$ 
        else
             $v \leftarrow \text{BESTCHILD}(v, C_p)$ 
    return  $v$ 
    
```

```

function DEFAULTPOLICY( $s$ )
    while  $s$  is non-terminal do
        choose  $a \in A(s)$  uniformly at random
         $s \leftarrow f(s, a)$ 
    return reward for state  $s$ 
    
```

```

function BESTCHILD( $v, c$ )
    return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}}$ 
```

```

function BACKUP( $v, \Delta$ )
    while  $v$  is not null do
         $N(v) \leftarrow N(v) + 1$ 
         $Q(v) \leftarrow Q(v) + \Delta(v, p)$ 
         $v \leftarrow \text{parent of } v$ 
    
```

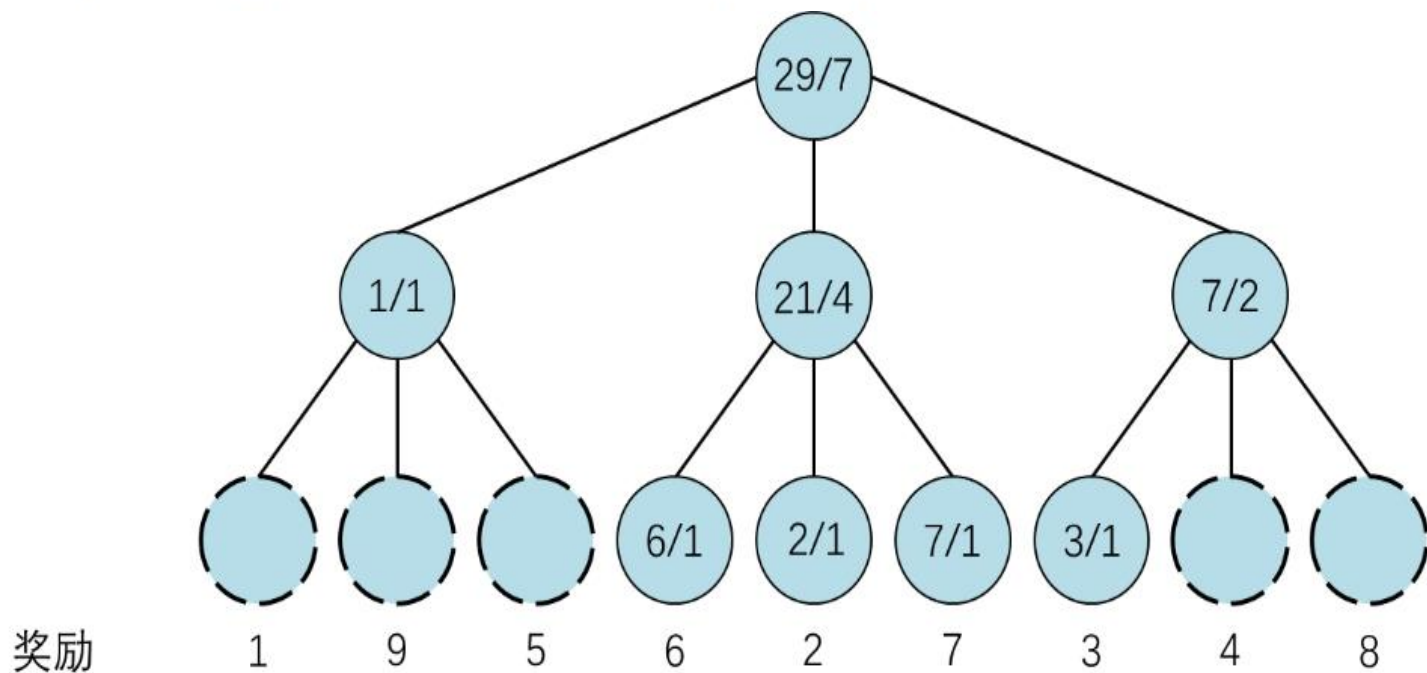
```

function EXPAND( $v$ )
    choose  $a \in$  untried actions from  $A(s(v))$ 
    add a new child  $v'$  to  $v$ 
        with  $s(v') = f(s(v), a)$ 
        and  $a(v') = a$ 
    return  $v'$ 
    
```

课上习题

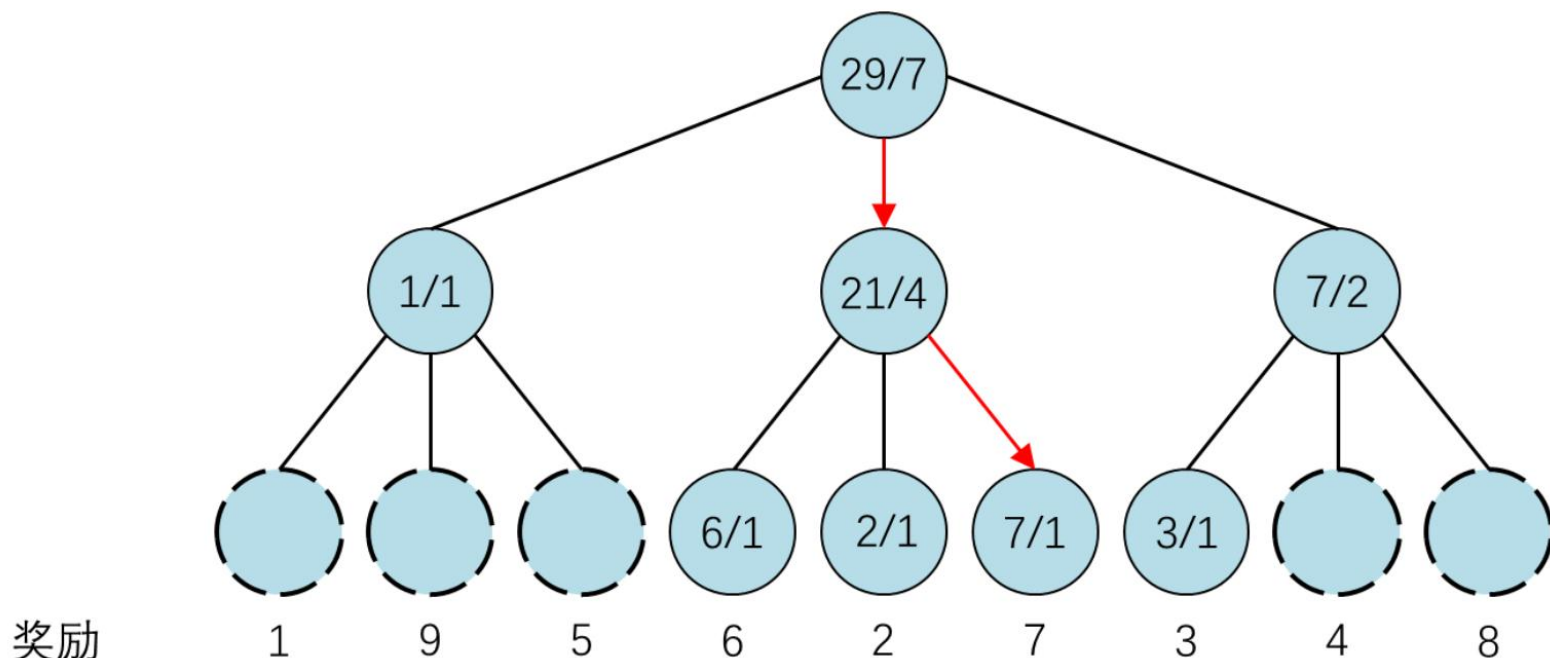
图 3 展示了一个蒙特卡洛树搜索的例子。其中每个叶子节点（终止节点）下标出了该节点对应的奖励。为了最大化取得的奖励，可利用蒙特卡洛树搜索求解奖励最大的路径。假设执行了若干步骤后，算法的状态如图 3 所示，节点内的数字分别表示“总奖励/访问次数”，虚线节点表示尚未扩展的节点。算法此时正要开始下一轮选择-扩展-模拟-反向传播的迭代

(1) 假设 UCB1 算法中的超参数 $C = 1$ ，请计算并画出算法选择过程经过的路径。



课上习题

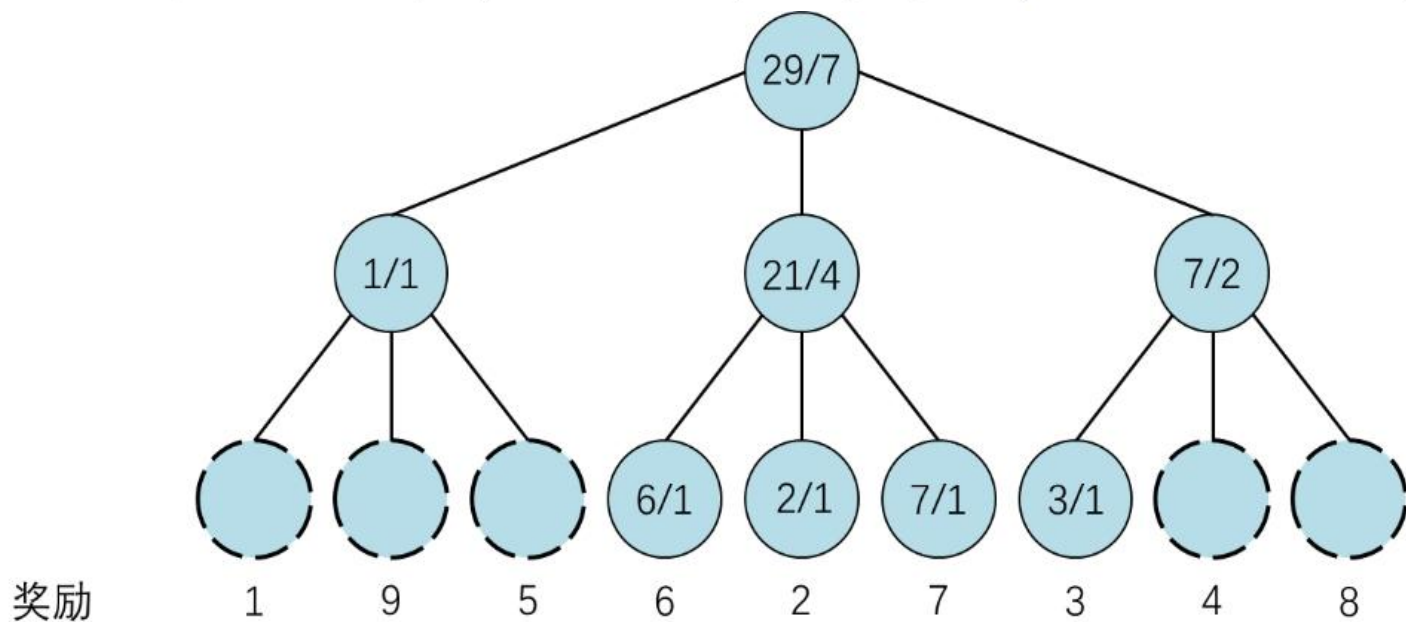
(1) 第一步三个节点的 UCB 值从左到右分别为 $\frac{1}{1} + \sqrt{\frac{2\ln 7}{1}} = 2.97$, $\frac{21}{4} + \sqrt{\frac{2\ln 7}{4}} = 6.24$, $\frac{7}{2} + \sqrt{\frac{2\ln 7}{2}} = 4.89$, 因此第一步选择第二层中间的节点。第二步三个节点的 UCB 值从左到右分别为 $\frac{6}{1} + \sqrt{\frac{2\ln 4}{1}} = 7.67$, $\frac{2}{1} + \sqrt{\frac{2\ln 4}{1}} = 3.67$, $\frac{7}{1} + \sqrt{\frac{2\ln 4}{1}} = 8.67$, 因此第二步选择奖励为 7 的节点。如下图所示。



课上习题

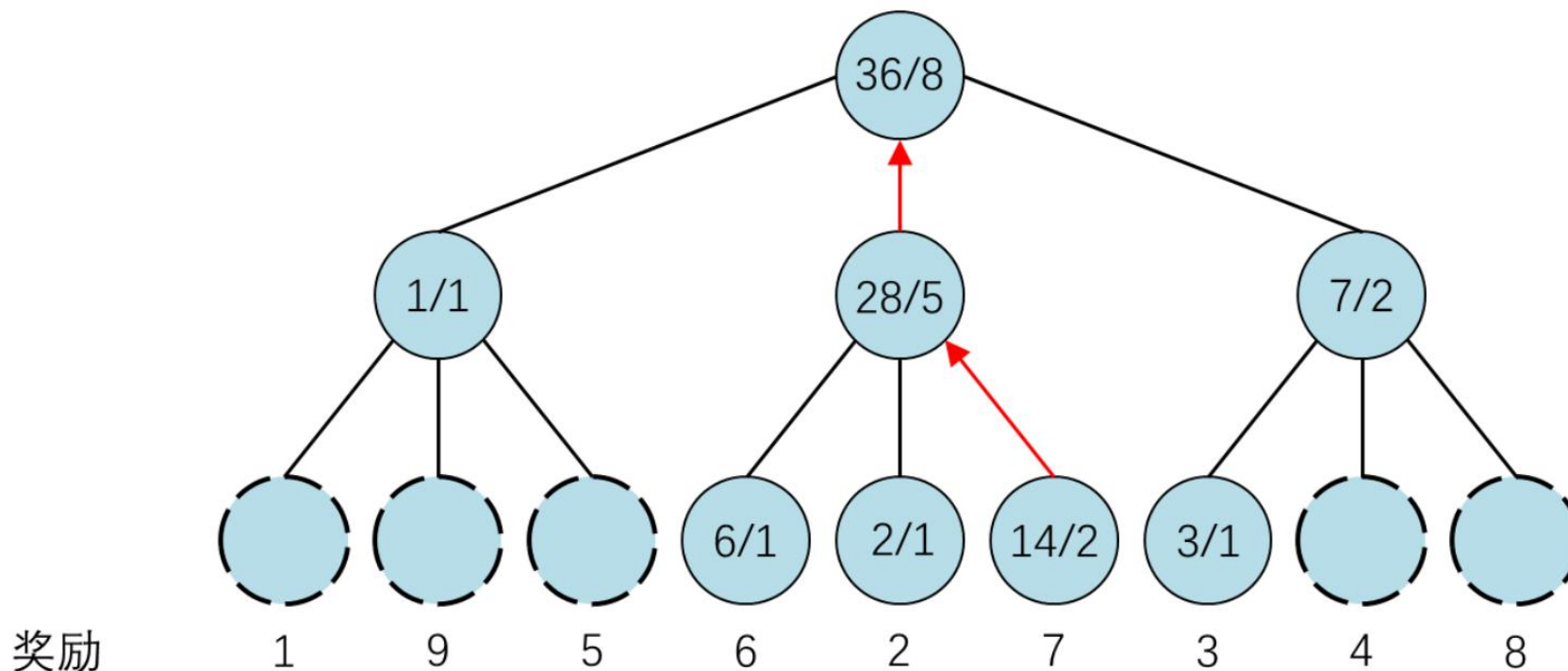
图 3 展示了一个蒙特卡洛树搜索的例子。其中每个叶子节点（终止节点）下标出了该节点对应的奖励。为了最大化取得的奖励，可利用蒙特卡洛树搜索求解奖励最大的路径。假设执行了若干步骤后，算法的状态如图 3 所示，节点内的数字分别表示“总奖励/访问次数”，虚线节点表示尚未扩展的节点。算法此时正要开始下一轮选择-扩展-模拟-反向传播的迭代

(2) 请继续执行扩展、模拟、反向传播步骤，并画出完成后的搜索树状态。（为了避免随机性，假设扩展总是扩展最左侧的未扩展节点，模拟总是选择最左侧的路径。）



课上习题

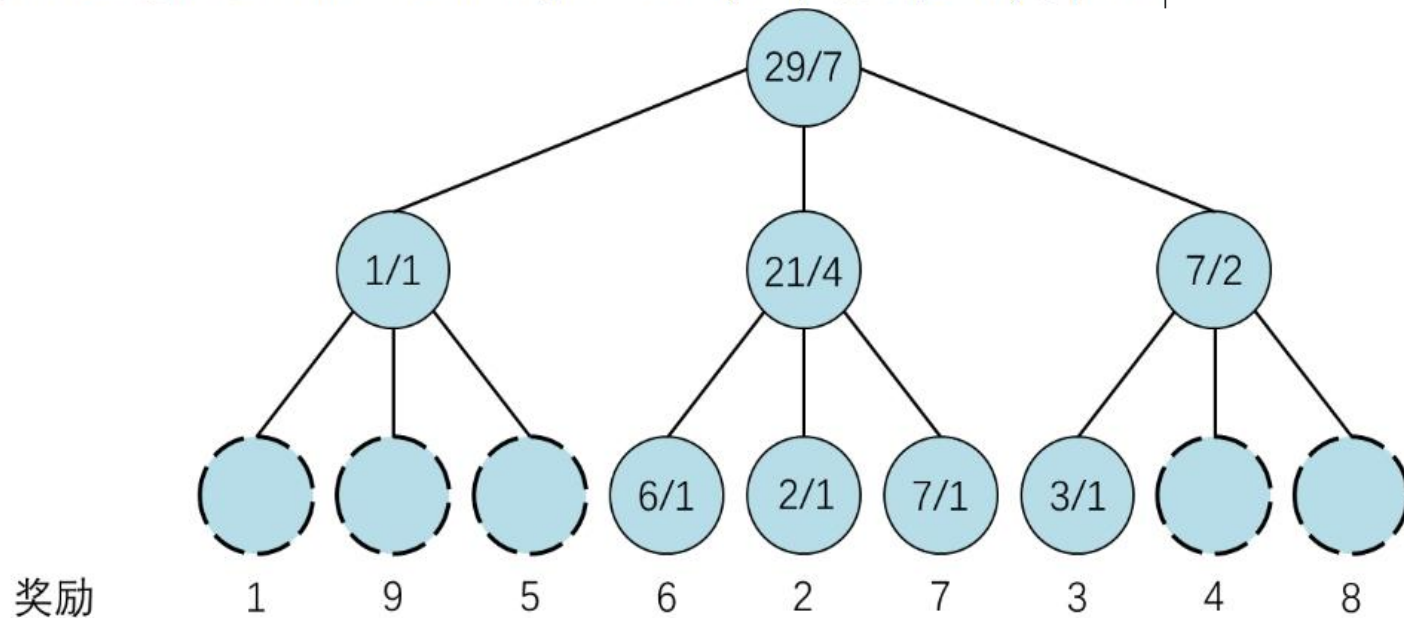
(2) 由于此时已经到达叶子节点，因此不需要进行扩展和模拟过程，反向传播后结果如下图所示



课上习题

图 3 展示了一个蒙特卡洛树搜索的例子。其中每个叶子节点（终止节点）下标出了该节点对应的奖励。为了最大化取得的奖励，可利用蒙特卡洛树搜索求解奖励最大的路径。假设执行了若干步骤后，算法的状态如图 3 所示，节点内的数字分别表示“总奖励/访问次数”，虚线节点表示尚未扩展的节点。算法此时正要开始下一轮选择-扩展-模拟-反向传播的迭代

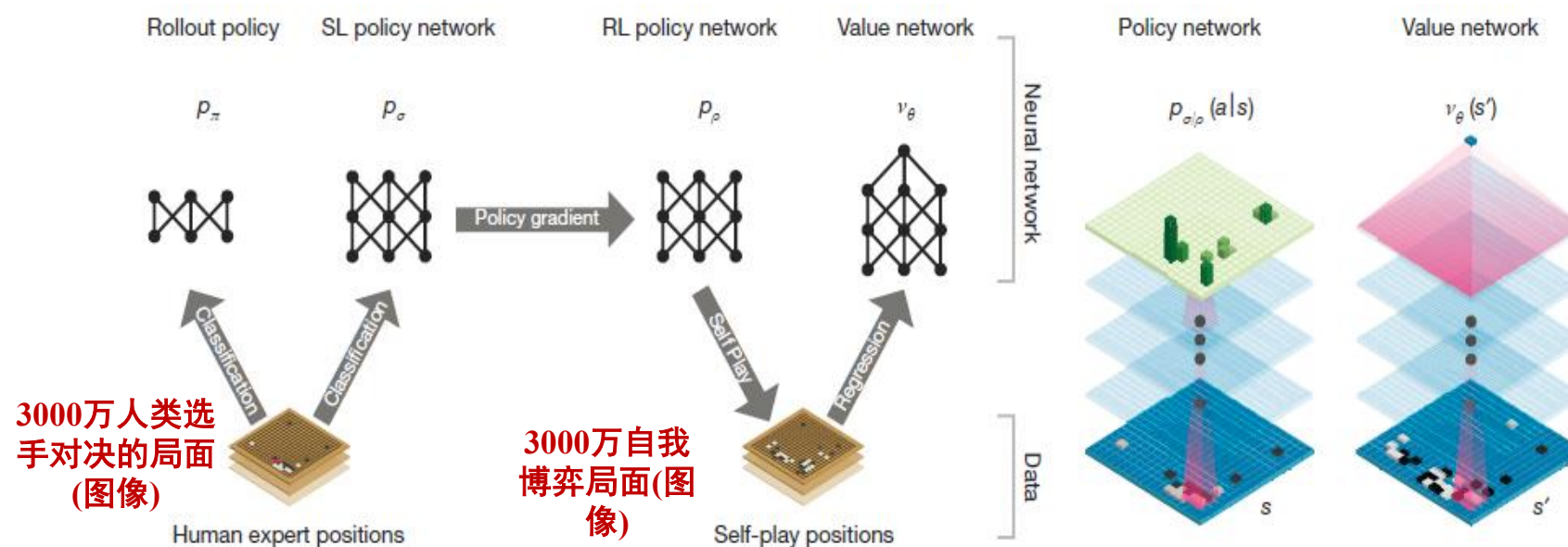
(3) 尝试进行若干次迭代，请问此时算法是否有效地找到奖励最大的叶子结点（奖励为 9），那么进行足够多次迭代以后又如何？如果希望提高算法的效率，应该做出怎样的调整？



- (3) 算法在很长一段时间内都会选择奖励为 7 的节点，而不会探索奖励为 9 的节点。当实验次数足够多时，第二层左侧的节点的 UCB 值最终会超过第二层中间节点的 UCB 值，因此只要实验次数足够多，算法是有可能探索到奖励为 9 的节点的。如果希望提高算法的效率，可考虑加大探索的力度，即取一个更大的超参数 C 。不难验证，在原题中的状态下，取 $C = 10$ 即可令算法选择第二层左侧的节点。

AlphaGo算法解读

- 将每个状态(局面)均视为一幅图像
 - 训练策略(policy)网络和价值(value)网络
 - $p_{\sigma,p}(a|s)$ 表示当前状态为 s (局面)时, 采取行动 a 后所得到的概率;
 $v_{\theta}(s')$ 表示当前状态为 s' 时, 整盘棋获胜的概率。



D. Silver, et.al., Mastering the game of Go with Deep Neural Networks and Tree Search, *Nature*, 529:484-490,2016

AlphaGo算法解读：策略网络的训练

- 基于监督学习来先训练策略网络

- Idea: perform supervised learning (SL) to predict human moves
- Given state s , predict probability distribution over moves a , $p_{\sigma,p}(a|s)$
- Trained on 30M positions, 57% accuracy on predicting human moves
- Also train a smaller, faster rollout policy network (24% accurate)

- 再基于强化学习来训练策略网络

- Idea: fine-tune policy network using reinforcement learning (RL)
- Initialize RL network to SL network
- Play two snapshots of the network against each other, update parameters to maximize expected final outcome
- RL network wins against SL network 80% of the time, wins against open-source Pachi Go program 85% of the time

AlphaGo算法解读：价值网络的训练

- Value network

- Idea: train network for position evaluation
- Given state s' , estimate $v_{\theta}(s')$, expected outcome of play starting with position s and following the learned policy for both players
- Train network by minimizing mean squared error between actual and predicted outcome
- Trained on 30M positions sampled from different self-play games

AlphaGo算法解读：策略网络和价值网络

- 在通过深度学习得到的策略网络和价值网络帮助之下，如下完成棋局局面的选择和搜索。给定节点 v_0 ，将具有如下最大值的节点 v 选择作为 v_0 的后续节点

$$\frac{Q(v)}{N(v)} + \frac{P(v|v_0)}{1 + N(v)}$$

- 这里 $P(v|v_0)$ 的值由策略网络计算得到。
- 在模拟策略阶段(default policy)，AlphaGo不仅考虑仿真结果，而且考虑价值网络计算结果。
- 策略网络和价值网络是离线训练得到的。

AlphaGo算法解读：策略网络和价值网络

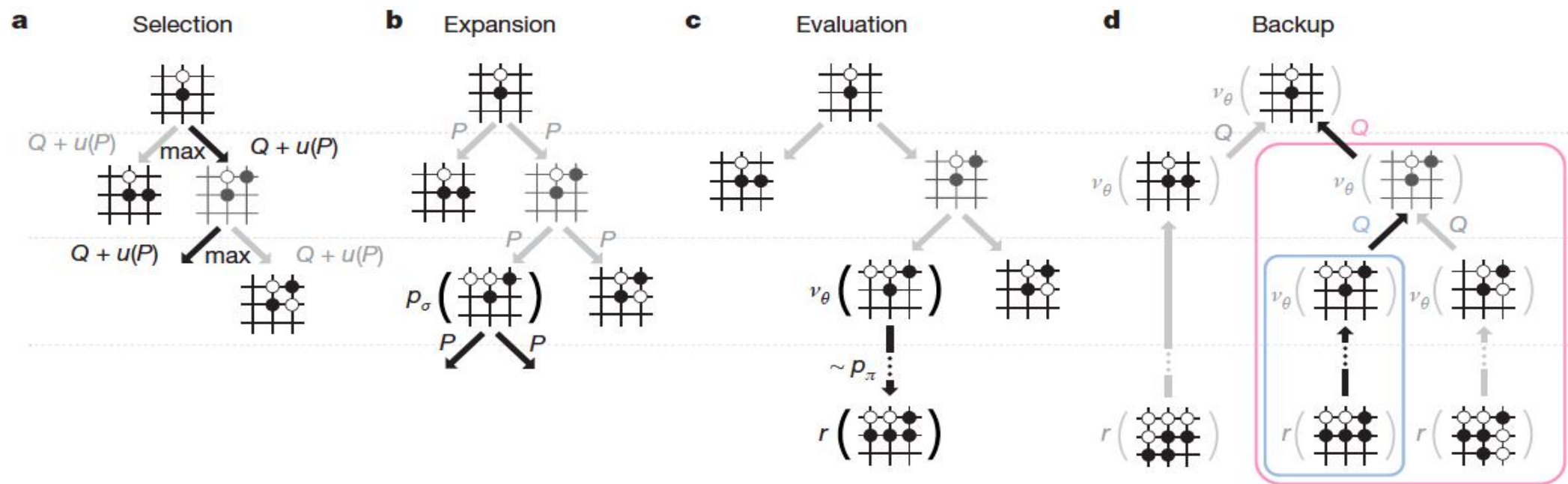


Figure 3 | Monte Carlo tree search in AlphaGo. a, Each simulation traverses the tree by selecting the edge with maximum action value Q , plus a bonus $u(P)$ that depends on a stored prior probability P for that edge. b, The leaf node may be expanded; the new node is processed once by the policy network p_σ and the output probabilities are stored as prior probabilities P for each action. c, At the end of a simulation, the leaf node

is evaluated in two ways: using the value network v_θ ; and by running a rollout to the end of the game with the fast rollout policy p_π , then computing the winner with function r . d, Action values Q are updated to track the mean value of all evaluations $r(\cdot)$ and $v_\theta(\cdot)$ in the subtree below that action.

价值网络计算

策略网络计算

$$a_t = \underset{a}{\operatorname{argmax}} (Q(s_t, a) + u(s_t, a))$$

$$N(s, a) = \sum_{i=1}^n 1(s, a, i)$$
$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^n 1(s, a, i) V(s_L^i)$$

$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)}$$

AlphaGo算法解读

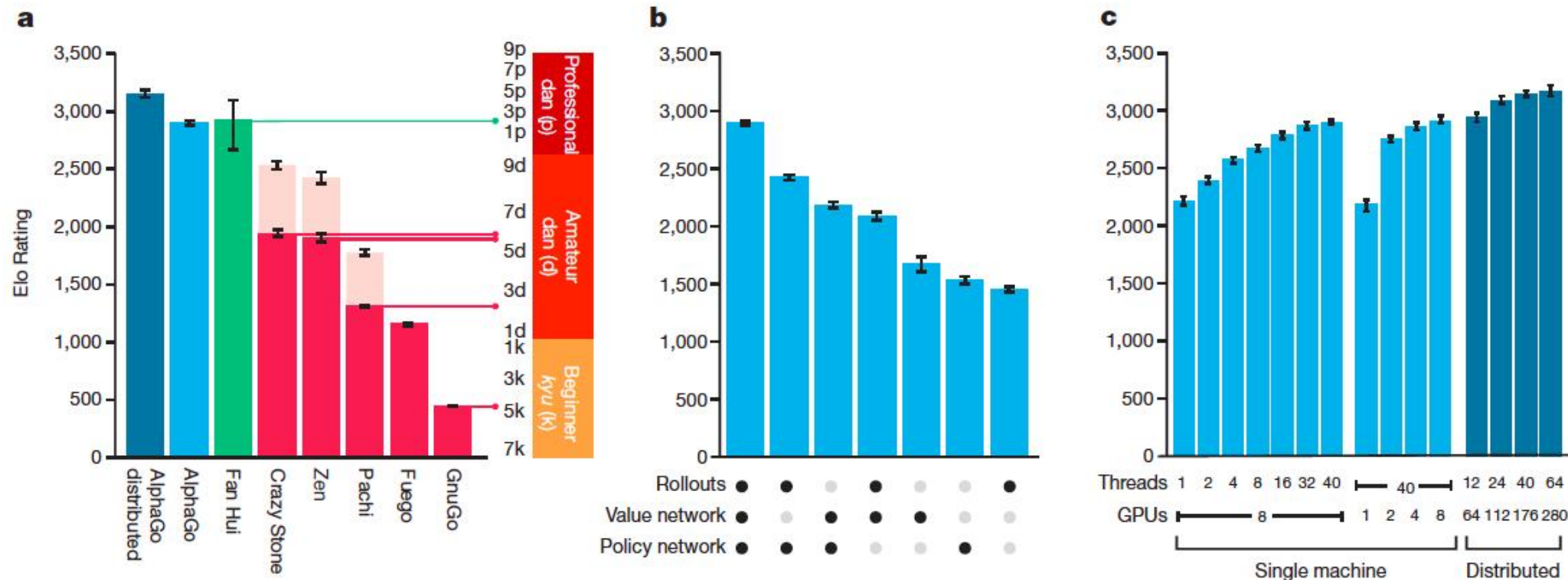
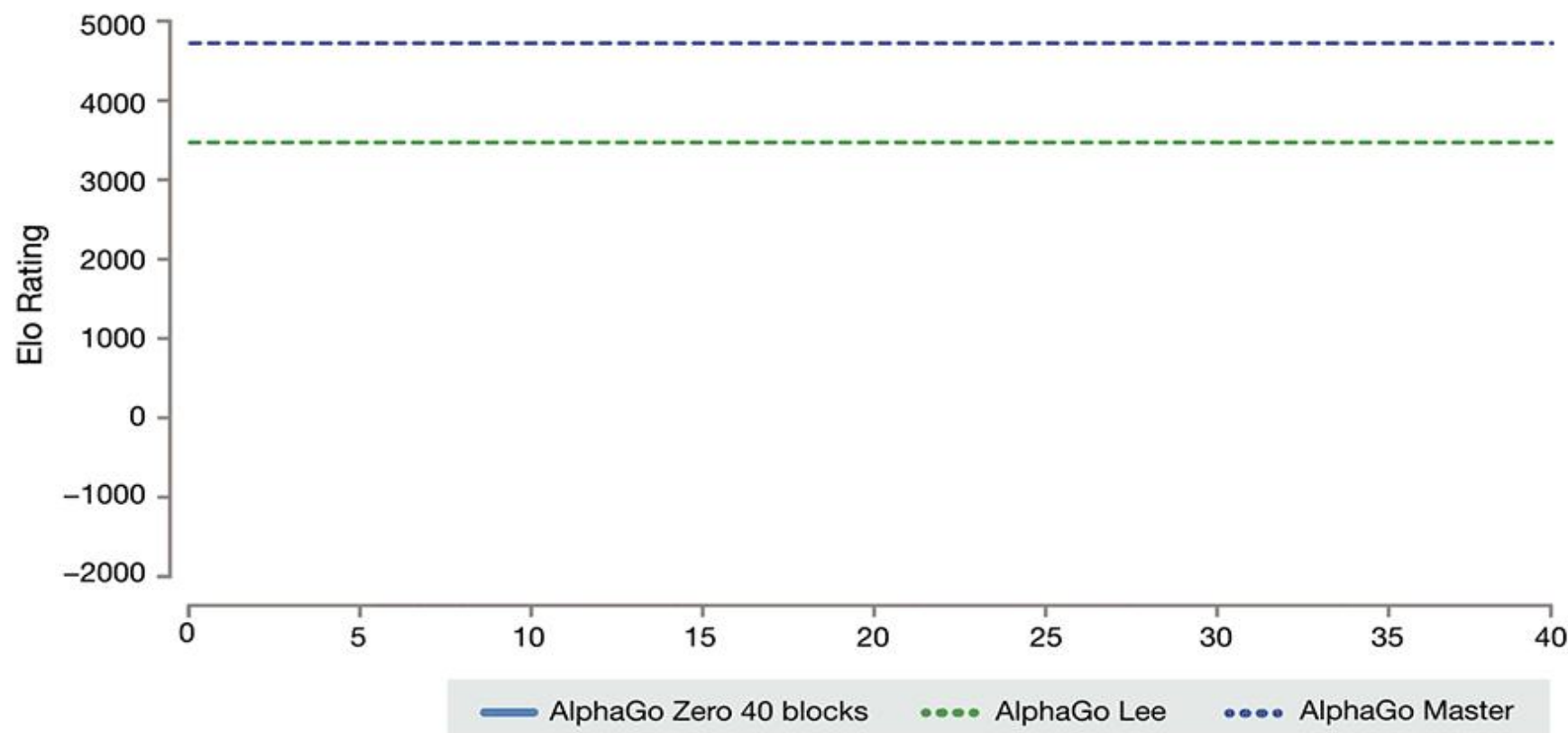


Figure 4 | Tournament evaluation of AlphaGo. **a**, Results of a tournament between different Go programs (see Extended Data Tables 6–11). Each program used approximately 5 s computation time per move. To provide a greater challenge to AlphaGo, some programs (pale upper bars) were given four handicap stones (that is, free moves at the start of every game) against all opponents. Programs were evaluated on an Elo scale³⁷: a 230 point gap corresponds to a 79% probability of winning, which roughly corresponds to one amateur *dan* rank advantage on KGS³⁸; an approximate correspondence to human ranks is also shown,

horizontal lines show KGS ranks achieved online by that program. Games against the human European champion Fan Hui were also included; these games used longer time controls. 95% confidence intervals are shown. **b**, Performance of AlphaGo, on a single machine, for different combinations of components. The version solely using the policy network does not perform any search. **c**, Scalability study of MCTS in AlphaGo with search threads and GPUs, using asynchronous search (light blue) or distributed search (dark blue), for 2 s per move.

AlphaGo算法解读：AlphaGo Zero (一张白纸绘蓝图)

- 经过40天训练后，Zero总计运行约2900万次自我对弈，得以击败AlphaGo Master，比分为89比11



Mastering the game of Go without human knowledge, *Nature*, volume 550, pages 354–359, 2017

拓展学习 (AlphaGo)

- https://www.bilibili.com/video/BV1hb4y197he/?spm_id_from=333.337.search-card.all.click
- https://www.bilibili.com/video/BV1Dh411Q77P/?spm_id_from=333.788.recommend_more_video.-1

阿尔法狗 | AlphaGo 万字解读 【下集】

5844播放 · 总弹幕数12 2021-03-20 13:20:01



谢谢!